



EGE ÜNİVERSİTESİ

LİSANS TEZİ

**MCP TABANLI YAPAY ZEKÂ KİŞİSEL ASİSTAN
PLATFORMU:
GOOGLE VE NOTION ENTEGRASYONU İLE
ÇOK KULLANICILI MİMARİ TASARIMI VE
GELİŞTİRİLMESİ**

**Mustafa Yiğit GÜZEL
Meriç ÖZKAYAĞAN**

Tez Danışmanı: Prof. Dr. Hasan BULUT
Bilgisayar Mühendisliği Anabilim Dalı
Sunuş Tarihi: Mayıs 2026
Bornova-İZMİR
2026

ÖZET

MCP TABANLI YAPAY ZEKÂ KİŞİSEL ASİSTAN PLATFORMU: GOOGLE VE NOTION ENTEGRASYONU İLE ÇOK KULLANICILI MİMARİ TASARIMI VE GELİŞTİRİLMESİ

GÜZEL, Mustafa Yiğit; ÖZKAYAĞAN, Meriç

Lisans Tezi, Bilgisayar Mühendisliği Bölümü

Tez Danışmanı: Prof. Dr. Hasan BULUT

Mayıs 2026, 55 sayfa

Bu çalışmada, Model Context Protocol (MCP) mimarisinden esinlenilerek tasarlanan çok kullanıcı bir yapay zekâ kişisel asistan platformu geliştirilmiştir. Platform, “Bring Your Own Key” (BYOK) modeli ile kullanıcıların kendi OpenAI veya Anthropic API anahtarlarını güvenli biçimde kullanmalarına ve Google Calendar, Google Drive, Gmail ile Notion hizmetlerini OAuth 2.0 aracılığıyla kendi hesaplarına bağlamalarına olanak tanımaktadır. Ek olarak Tavily API üzerinden güncel bilgiye erişim sağlayan bir web arama aracı entegre edilmiştir. Sistem; Fastify tabanlı RESTful arka uç, Next.js ile geliştirilmiş ön yüz ve PostgreSQL veritabanından oluşan monorepo yapıda kurgulanmış; hassas veriler (API anahtarları ve OAuth token’ları) AES-256-GCM algoritması ile şifrelenmiştir. Araç yönetim katmanında, MCP’nin araç kayıt defteri yaklaşımı benimsenmiş, toplam 13 araç ToolDefinition arayüzüne uygun şekilde tanımlanmıştır. Yapay zekâ modeli, Vercel AI SDK’nın çok adımlı akış (streaming) desteğiyle, doğal dil komutlarına karşılık bu araçları ardışık ve gerçek zamanlı olarak çağırabilmektedir. Güvenlik katmanı; JWT tabanlı oturum, bcrypt parola karma, helmet HTTP başlıkları, rate-limit ve OAuth akışında JWT imzalı state parametresi ile çok seviyeli olarak tasarlanmıştır. Token süreleri izlenmekte, bitime beş dakikadan az kalan Google access token’ları otomatik olarak yenilenmektedir. Prototip, Docker Compose üzerinden üç servisli konteyner ortamında çalıştırılabilmekte; tüm API uç noktaları için otomatik entegrasyon test betikleri hazırlanmıştır.

Anahtar sözcükler: Yapay zekâ asistanı, Model Context Protocol, OAuth 2.0, araç çağırma, çok kullanıcı mimari, AES-256-GCM.

ABSTRACT

MCP-INSPIRED AI PERSONAL ASSISTANT PLATFORM: DESIGN AND DEVELOPMENT OF A MULTI-TENANT ARCHITECTURE WITH GOOGLE AND NOTION INTEGRATION

GÜZEL, Mustafa Yiğit; ÖZKAYAĞAN, Meriç

B.Sc. Thesis, Department of Computer Engineering

Supervisor: Prof. Dr. Hasan BULUT

May 2026, 55 pages

In this study, a multi-tenant artificial intelligence personal assistant platform inspired by the Model Context Protocol (MCP) architecture is designed and implemented. With a “Bring Your Own Key” (BYOK) model, the platform allows users to securely use their own OpenAI or Anthropic API keys and to connect Google Calendar, Google Drive, Gmail, and Notion to their own accounts via OAuth 2.0. In addition, a web search tool is integrated via the Tavily API to provide real-time knowledge access. The system follows a monorepo structure consisting of a Fastify-based RESTful backend, a Next.js frontend, and a PostgreSQL database; sensitive data (API keys and OAuth tokens) are encrypted with the AES-256-GCM algorithm. The tool-management layer adopts the MCP tool-registry paradigm, and a total of 13 tools are defined under the ToolDefinition interface. Using Vercel AI SDK’s multi-step streaming support, the AI model can invoke these tools sequentially and in real time in response to natural-language instructions. The security layer is multi-tier: JWT-based sessions, bcrypt password hashing, helmet HTTP headers, rate limiting, and a JWT-signed state parameter in the OAuth flow. Token lifetimes are tracked, and Google access tokens whose expiry is within five minutes are automatically refreshed. The prototype runs in a three-service Docker Compose environment and is validated by automated integration test scripts covering every API endpoint.

Keywords: AI assistant, Model Context Protocol, OAuth 2.0, tool calling, multi-tenant architecture, AES-256-GCM.

TEŞEKKÜR

Bu tez çalışması boyunca bilimsel rehberliği, değerli önerileri ve sabırla sağladığı akademik yönlendirme ile katkıda bulunan tez danışmanımız sayın **Prof. Dr. Hasan BULUT**'a teşekkürü bir borç biliriz. Ayrıca Ege Üniversitesi Bilgisayar Mühendisliği Bölümü öğretim üyelerine, lisans eğitimimiz boyunca edindiğimiz bilgi birikimi için şükranlarımızı sunarız. Son olarak, çalışma süresince verdikleri manevi destek için ailelerimize teşekkür ederiz.

İÇİNDEKİLER

	Sayfa
ÖZET	vii
ABSTRACT	ix
TEŞEKKÜR	xi
ŞEKİLLER DİZİNİ	xv
ÇİZELGELER DİZİNİ	xvii
SİMGELER VE KISALTMALAR DİZİNİ	xix
1. GİRİŞ	1
1.1 Çalışmanın Amacı	2
1.2 Çalışmanın Kapsamı	3
1.3 Tezin Organizasyonu	4
2. ÖNCEKİ ÇALIŞMALAR	5
2.1 Büyük Dil Modelleri ve Araç Çağırma	5
2.2 Model Context Protocol	6
2.3 Mevcut Yapay Zekâ Asistan Uygulamaları	7
2.4 OAuth 2.0 Tabanlı Üçüncü Parti Entegrasyonları	8
2.5 Vercel AI SDK ve Akış Tabanlı Uygulamalar	9
2.6 Web Arama Entegrasyonu	10
2.7 Bu Çalışmanın Farklılıkları	11

	Sayfa
3. YÖNTEM VE TEKNOLOJİLER	12
3.1 Yöntem	12
3.2 Kullanılan Teknolojiler	14
3.3 Monorepo Yapısı	16
4. SİSTEM TASARIMI	17
4.1 Gereksinim Analizi	17
4.2 Veritabanı Tasarımı	19
4.3 REST API Tasarımı	22
4.4 Güvenlik Tasarımı	25
4.5 Araç Yönetim Katmanı	28
5. GERÇEKLEŞTİRİM	30
5.1 Backend (Fastify)	30
5.2 Frontend (Next.js)	33
5.3 Paylaşılan Paketler	36
5.4 Sistem Mesajı Tasarımı	38
5.5 Docker Altyapısı	40
6. SINAMA VE DEĞERLENDİRME	41
6.1 Birim ve Entegrasyon Testleri	41
6.2 Güvenlik Değerlendirmesi	42
6.3 Token Yenileme Doğrulaması	43
6.4 Araç Çağırma Doğruluğu	44

	Sayfa
7. SONUÇ VE TARTIŞMA	45
8. ÖNERİLER	47
KAYNAKLAR DİZİNİ	49

ŞEKİLLER DİZİNİ

Şekil	Sayfa
3.1 Monorepo paket bağımlılıkları	16
4.1 Varlık-ilişki diyagramı	20
4.2 OAuth 2.0 yetkilendirme akışı	26
4.3 Araç çağırma ve yürütme sırası	28
5.1 Sohbet endpoint'inin akış diyagramı	31
5.2 Ön yüz sohbet arayüzü ekran görüntüsü	34
5.3 Docker Compose servis mimarisi	40
5.4 Kullanıcı onboarding akışı	41
5.5 Uçtan uca sohbet akışı (sequence)	43
5.6 LLM karar döngüsü (multi-step reasoning)	44
5.7 Tanıtım sayfası	45
5.8 Giriş sayfası	46
5.9 Ayarlar sayfası	47

Şekil		Sayfa
5.10	Sohbet ekranı (boş durum)	48
5.11	Sohbet ekranı (sorgu girilmiş)	49

ÇİZELGELER DİZİNİ

Çizelge		Sayfa
3.1	Kullanılan teknolojiler ve amaçları	15
4.1	Veritabanı tabloları ve açıklamaları	19
4.2	REST API uç noktaları	23
4.3	Kayıtlı MCP araçları	29

SİMGELER VE KISALTMALAR DİZİNİ

Kısaltmalar Açıklama

AES-256-GCM Advanced Encryption Standard – 256 bit, Galois/Counter Mode

API Application Programming Interface

BYOK Bring Your Own Key

CRUD Create, Read, Update, Delete

CORS Cross-Origin Resource Sharing

CSRF Cross-Site Request Forgery

GCM Galois/Counter Mode

HTTP / HTTPS HyperText Transfer Protocol / Secure

JSON JavaScript Object Notation

JWT JSON Web Token

LLM Large Language Model

MCP Model Context Protocol

OAuth Open Authorization

ORM Object-Relational Mapping

PAT Personal Access Token

REST Representational State Transfer

RFC Request For Comments

SDK Software Development Kit

SQL Structured Query Language

TLS Transport Layer Security

URL / URI Uniform Resource Locator / Identifier

UTC Coordinated Universal Time

UUID Universally Unique Identifier

1.5 Problem Tanımı ve Motivasyon

Günümüz bilgi işlem ortamında profesyoneller; e-posta, takvim, bulut depolama ve not alma uygulamaları arasında sürekli bağlam değiştirmek (context switching) zorunda kalmaktadır. Mark, Gudith ve Klocke (2008), kesintili çalışmanın verimlilik kaybına ve stres artışına yol açtığını ampirik olarak göstermiştir. Yapay zekâ destekli kişisel asistanlar, bu parçalı deneyimi tek bir doğal dil arayüzünde birleştirme potansiyeli taşımaktadır.

Mevcut ticari asistanlar (ChatGPT, Gemini, Microsoft Copilot) güçlü dil anlama yetenekleri sunmakla birlikte, kullanıcının kişisel verilerine erişim konusunda sınırlı veya kapalı ekosistemlerdir. Kurumsal kullanıcılar veri egemenliği endişeleri taşıırken; bireysel kullanıcılar kendi API anahtarlarını kontrol etmek ve tercih ettikleri LLM sağlayıcısını seçmek istemektedir. Bu gereksinimler, BYOK ve OAuth tabanlı açık entegrasyon mimarisini motive etmektedir.

Model Context Protocol (MCP), Anthropic tarafından 2024 yılında tanıtılan ve yapay zekâ modellerinin harici araç ve veri kaynaklarıyla standart biçimde iletişim kurmasını hedefleyen bir protokoldür. MCP'nin araç soyutlama fikri, bu çalışmanın mimari temelini oluşturmaktadır. Ancak tam protokol uygulaması yerine, MCP'den esinlenen hafif bir registry deseni benimsenerek geliştirme hızı ve dağıtım basitliği önceliklendirilmiştir.

1.6 Araştırma Soruları

Bu tez çalışması aşağıdaki araştırma sorularına yanıt aramaktadır: (RQ1) MCP esinli bir araç kayıt defteri, çok kullanıcıli bir ortamda güvenli OAuth token yönetimi ile birleştirilebilir mi? (RQ2) BYOK modeli ile kullanıcı API anahtarları veritabanında AES-256-GCM ile güvenli biçimde saklanabilir mi? (RQ3) Vercel AI SDK'nın maxSteps özelliği, doğal dil komutlarını çok adımlı araç zincirlerine dönüştürmek için yeterli midir? (RQ4) Sistem mesajı tabanlı prompt mühendisliği, araç seçim doğruluğunu kabul edilebilir düzeye (%80+) çıkarabilir mi?

Bu sorular, çalışmanın tasarım kararlarını ve değerlendirme metriklerini yönlendirmiştir. RQ1-RQ3 mimari ve uygulama bölümlerinde; RQ4 ise Bölüm 6'daki araç çağırma doğruluğu testleri ile yanıtlanmıştır.

1.7 Katkılar

Çalışmanın bilimsel ve mühendislik katkıları şu şekilde özetlenebilir: (K1) MCP kavramının OAuth 2.0, AES-256-GCM şifreleme ve JWT kimlik doğrulaması ile entegre edildiği uçtan uca bir referans mimari sunulması; (K2) BYOK modelinin çok kiracılı SaaS ortamında uygulanabilir bir desen olarak gösterilmesi; (K3) Gmail okuma/gönderme ayrımı ve Notion mükerrer sayfa önleme gibi somut prompt mühendisliği kalıplarının dokümente edilmesi; (K4) 13 araçluk açık kaynak bir monorepo referans implementasyonunun paylaşılması.

Pratik katkı açısından, geliştirilen platform docker compose up --build komutu ile tek adımda çalıştırılabilir durumdadır. Bu, benzer projelere başlayacak geliştiriciler için tekrarlanabilir bir başlangıç noktası sağlamaktadır.

Yapay zekâ kişisel asistanları, son beş yılda Siri ve Alexa gibi kural tabanlı sistemlerden, GPT-4 ve Claude gibi büyük dil modellerine (LLM) dayalı sistemlere evrilmiştir. LLM'ler genel amaçlı dil anlama ve üretme kapasitesi sunmakla birlikte, gerçek zamanlı veri erişimi ve eylem gerçekleştirme (action taking) yetenekleri sınırlıdır. Araç çağırma (tool calling) mekanizması bu boşluğu doldurmaktadır.

Çalışmanın kapsamı bilinçli olarak sınırlandırılmıştır. Sesli asistan, mobil uygulama, RAG tabanlı belge sorgulama, fine-tuning ve çoklu dil desteği bu tez kapsamı dışındadır. Odak noktası; web tabanlı, çok kullanıcı, OAuth entegreli ve MCP esinli araç yönetimine sahip bir prototip platformun tasarımı, geliştirilmesi ve değerlendirilmesidir.

Tez organizasyonu şu şekildedir: Bölüm 2'de ilgili literatür incelenmekte; Bölüm 3'te yöntem ve teknolojiler açıklanmakta; Bölüm 4'te sistem tasarımı detaylandırılmaktadır; Bölüm 5'te gerçekleştirim anlatılmakta; Bölüm 6'da sinama ve değerlendirme sunulmakta; Bölüm 7'de sonuç ve tartışma yapılmakta; Bölüm 8'de gelecek çalışma önerileri belirtilmektedir.

1.8 Tanımlar ve Terminoloji

Bu tez kapsamında sık kullanılan terimler şu anlamlarda kullanılmaktadır. Büyük Dil Modeli (LLM): geniş metin veri kümeleri üzerinde eğitilmiş, doğal dil anlama ve üretme yeteneğine sahip yapay zekâ modeli. Araç çağırma (tool calling / function calling): LLM'in yapılandırılmış JSON çıktısı ile harici fonksiyonları tetikleme. Model Context Protocol (MCP): Anthropic tarafından tanımlanan, yapay zekâ modellerinin harici araç ve kaynaklarla iletişim protokolü. BYOK (Bring Your Own Key): kullanıcının kendi API anahtarını platforma sağlama modeli. OAuth 2.0: üçüncü parti uygulamaların kullanıcı adına sınırlı erişim elde etmesini sağlayan yetkilendirme çerçevesi. Çok kiracılı (multi-tenant): tek uygulama örneğinin birden fazla bağımsız kullanıcıya hizmet vermesi.

Akış (streaming): LLM yanıtının tamamlanmasını beklemeden token token iletilmesi. Registry (kayıt defteri): araç tanımlarının merkezi olarak depolandığı ve ad ile sorgulandığı yazılım deseni. Repository: veritabanı erişim mantığını kapsülleyen veri erişim katmanı deseni. Monorepo: birden fazla paketin tek versiyon kontrol deposunda yönetildiği proje organizasyon yapısı.

1. GİRİŞ

Dijitalleşme ile birlikte bireylerin ve profesyonellerin iş akışlarını yönetmek için başvurdukları araçların sayısı her geçen yıl artmaktadır. Günümüzde bir bilgi çalışan; toplantılarını Google Calendar, dosyalarını Google Drive, e-postalarını Gmail, notlarını ise Notion gibi platformlarda tutmaktadır. Bu araçlar tek tek güçlü olmakla birlikte, kullanıcıyı sürekli olarak platform değiştirmeye zorlayan bir “bağlam anahtarlama” (context switching) maliyeti doğurmaktadır. Yapılan çalışmalar, bu maliyetin iş stresini artırdığını ve odaklanma süresini kısalttığını göstermektedir (Mark vd., 2008).

Paralel olarak, büyük dil modellerindeki (LLM) son ilerlemeler, yapay zekâ sistemlerinin yalnızca metin üretmekle kalmayıp, tanımlı işlevleri çağırarak dış dünya ile etkileşime geçebildiği bir paradigmayı doğurmuştur. Bu yetenek “araç çağırma” (tool calling) veya “fonksiyon çağırma” olarak bilinmektedir ve ilk olarak OpenAI tarafından GPT modellerine eklenmiştir (OpenAI, 2023). Anthropic’in Claude modelleri de benzer bir yeteneği desteklemektedir (Anthropic, 2024b).

Kasım 2024’te Anthropic tarafından duyurulan **Model Context Protocol (MCP)**, yapay zekâ uygulamalarını veri kaynaklarına bağlamak için evrensel ve açık bir standart önermektedir (Anthropic, 2024a). MCP, istemci-sunucu mimarisi ile araçların, kaynakların ve bağlam bilgilerinin standart bir arayüz üzerinden sunulmasını tanımlamaktadır. Protokolün iki temel katkısı, araç tanımlarının tekdüze bir sözleşme ile ifade edilmesi ve bu tanımların ayrı bir “sunucu” tarafından barındırılarak istemciden bağımsızlaşmasıdır.

Bu lisans tezinde, MCP mimarisinden esinlenilerek tasarlanmış, çok kullanıcılı bir yapay zekâ kişisel asistan platformu tasarlanmış ve prototipi geliştirilmiştir. Mevcut çözümlerin aksine platformumuz; (i) kullanıcıya kendi LLM API anahtarını kullanma imkânı sağlayan BYOK modelini, (ii) kullanıcı verisinin üçüncü parti platformlarda kalmasına özen gösteren OAuth 2.0 tabanlı entegrasyonları ve (iii) her kullanıcının verisini AES-256-GCM ile şifreleyen çok kiracılı bir güvenlik katmanını bir arada sunmaktadır.

Yapay zekâ tabanlı kişisel asistanların yaygınlaşmasıyla birlikte, kullanıcıların kendi üretkenlik araçlarıyla bu sistemleri entegre etme talebi artmıştır. Ancak mevcut çözümlerin büyük çoğunluğu, kullanıcı verisini sağlayıcının yönetilen altyapısı üzerinde işlemekte ve API anahtarı, OAuth token gibi hassas bilgilerin saklanması üçüncü tarafa bırakmaktadır. Bu durum, hem bireysel kullanıcılar hem de kurumsal kullanıcılar için veri sahipliği, gizlilik ve maliyet öngörülebilirliği açısından önemli bir endişe kaynağı oluşturmaktadır.

Bu çalışma; söz konusu endişeleri "Bring Your Own Key" (BYOK) modeli, açık kaynak dağıtımı ve yerel altyapıda çalıştırılabilen mimari ile bir araya getirerek ele almaktadır. Geliştirilen platform, kullanıcının kendi sağlayıcı kimlik bilgilerini taşıyabildiği, hassas verilerin platform tarafından şifreli olarak saklandığı ve birden fazla üçüncü parti hizmetin tek bir sohbet arayüzünde toplandığı bir referans tasarımı önermektedir.

1.1 Çalışmanın Amacı

Bu çalışmanın temel amaçları şu şekilde özetlenebilir:

1. Kullanıcıların kendi LLM API anahtarlarını güvenli biçimde saklayabildiği bir BYOK mimarisi tasarlamak.
2. Google Calendar, Google Drive, Gmail ve Notion hizmetlerini OAuth 2.0 ile entegre ederek tek bir sohbet arayüzünden yönetilebilir hâle getirmek.
3. Güncel bilgi erişimi için Tavily tabanlı bir web arama aracı entegre etmek.
4. Yapay zekâ modelinin doğal dil komutları ile bu 13 aracı ardışık biçimde çağırabilmesini sağlamak.
5. Hassas verilerin veritabanında yalnızca şifreli biçimde saklandığı, kimlik doğrulaması JWT ile yapılan, rate-limit ve güvenlik başlıklarıyla donatılmış çok seviyeli bir güvenlik katmanı inşa etmek.
6. Geliştirilen sistemin Docker Compose üzerinden tek komutla çalıştırılabildiği, bakımı ve yeni araçlarla genişletilmesi kolay bir kod tabanı üretmek.

1.2 Çalışmanın Kapsamı

Çalışma kapsamında gerçekleştirilen başlıca iş paketleri şunlardır: gereksinim ve araç analizi; ilişkisel veritabanı şeması tasarımı; Fastify tabanlı RESTful arka uç geliştirme; Next.js tabanlı ön yüz geliştirme; MCP esinli araç kayıt katmanı; AES-256-GCM şifreleme ile hassas veri saklama; OAuth 2.0 akışları (Google ve Notion) ile otomatik token yenileme; sohbet, araç sonuç görselleştirme, sesli giriş ve dosya ekleme özellikleriyle tam işlevli bir ön yüz arayüzü; Docker Compose ile konteyner tabanlı dağıtım ve uçtan uca entegrasyon test betikleri.

1.3 Tezin Organizasyonu

Tezin geri kalanı şu şekilde organize edilmiştir: **Bölüm 2**'de literatür taraması sunulmaktadır. **Bölüm 3**'te kullanılan yöntem ve teknolojiler özetlenmektedir. **Bölüm 4**'te

sistemin tasarımı, **Bölüm 5**'te gerçekleştirim detayları anlatılmaktadır. **Bölüm 6**, sına ve değerlendirme çalışmalarına ayrılmıştır. **Bölüm 7**'de sonuçlar tartışılmakta, **Bölüm 8**'de gelecek çalışmalar için öneriler sunulmaktadır.

1.4 Tezin Akademik Konumu

Bu tez çalışması, yazılım mühendisliği disiplininin birden fazla alt alanını kesiştiren bir çalışma olarak konumlanmaktadır. Birinci olarak, dağıtık sistemler ve servis odaklı mimari (SOA) alanlarında çok kiracılı (multi-tenant) bir SaaS uygulamasının tasarımına dair pratik bir örnek sunmaktadır. İkinci olarak, uygulamalı kriptografi alanında AES-256-GCM gibi yaygın bir sözleşmenin (authenticated encryption) gerçek bir üretim senaryosunda nasıl uygulanacağına dair somut bir örnek üretmektedir. Üçüncü olarak, doğal dil işleme ve yapay zekâ alanında, büyük dil modellerinin araç çağırma yetenekleriyle dış sistemlere bağlanmasının mühendislik gereklilikleri üzerine bir vaka çalışması sağlamaktadır.

Söz konusu disiplinler arası konum, çalışmanın üretim ortamına yakın bir prototip ortaya koymasını gerekli kılmıştır. Yazılım mimarisi tarafında sadece bir kavram doğrulaması (proof-of-concept) değil; testleri yazılmış, Docker üzerinde dağıtılabilen, hata yönetimi ve loglama gibi olgun yazılım pratikleri uygulanmış bir kod tabanı üretilmiştir. Yapay zekâ tarafında, mevcut modellerin (GPT-4o, Claude Haiku 4.5) doğrudan kullanılması; çalışmanın katkısının yeni bir model eğitmek değil, mevcut modelleri bir sistem mimarisi içinde verimli biçimde organize etmek olduğunu göstermektedir.

2.8 Çok Kiracılı SaaS Mimarileri

Çok kiracılı (multi-tenant) yazılım mimarileri, tek bir uygulama örneğinin birden fazla müşteriye (kiracıya) hizmet vermesini ifade eder. Veri izolasyonu stratejileri üç ana grupta incelenmektedir: paylaşımlı şema (shared schema, tenant_id sütunu ile), şema başına kiracı (schema per tenant) ve veritabanı başına kiracı (database per tenant). Geliştirilen platform, paylaşımlı şema yaklaşımını benimsemekte; tüm tablolarda user_id foreign key ile kiracı ayrımı sağlamaktadır.

BYOK (Bring Your Own Key) modeli, özellikle yapay zekâ SaaS uygulamalarında son yıllarda yaygınlaşmaktadır. OpenRouter ve LiteLLM gibi ara katmanlar, kullanıcıların kendi API anahtarlarını proxy üzerinden kullanmasına olanak tanır. Bu çalışmadaki fark, BYOK'un doğrudan uygulama veritabanında AES-256-GCM ile şifrlenmesi ve OAuth token'larıyla birlikte bütünleşik bir güvenlik katmanında yönetilmesidir.

2.9 Prompt Mühendisliği ve Araç Seçimi

Büyük dil modellerinin araç çağırma doğruluğu, sistem mesajının (system prompt) kalitesine doğrudan bağlıdır. Wei vd. (2022) chain-of-thought prompting'in akıl yürütme performansını artırdığını göstermiştir; benzer biçimde, araç seçiminde açık karar ağaçları (decision trees) modelin doğru fonksiyonu çağırma olasılığını yükseltmektedir. Bu çalışmada buildSystemPrompt fonksiyonu, Gmail, Notion ve Calendar için ayrı karar ağaçları içermektedir.

Yanlış pozitif araç çağrılarını (örneğin e-posta okuma isteğinde gönderme aracının çağırılması) ciddi kullanıcı deneyimi sorunlarına yol açabilir. Sistem mesajında CRITICAL TOOL SELECTION RULES bölümü, bu riski azaltmak için her araç çağırısı öncesinde modele hatırlatma görevi görür. Bu yaklaşım, fine-tuning veya RLHF gerektirmeden davranışsal doğruluğu artırmaktadır.

2.10 Açık Kaynak Yapay Zekâ Asistan Platformları

Open WebUI (2026), yerel LLM çalıştırma, RAG (Retrieval-Augmented Generation) ve eklenti desteği sunan popüler bir açık kaynak projedir. Ollama ve OpenAI uyumlu API'lerle çalışabilir; ancak yerleşik OAuth tabanlı Google Workspace entegrasyonu bulunmamaktadır. LibreChat ve LobeChat benzer konumdadır; odak noktaları model esnekliği ve arayüz özelleştirmesidir.

Bu çalışmanın açık kaynak asistanlardan temel farkı, MCP esinli araç registry'sinin OAuth token yenileme, şifreleme ve kullanıcı izolasyonu ile entegre edilmiş olmasıdır. Araçlar yalnızca LLM'e tanıtılmakla kalmaz; her execute çağırısı userId ile kapsüllenir ve ensureValidToken ile güncel yetkilendirme sağlanır.

OpenAI'nin 2023 yılında duyurduğu function calling (araç çağırma) özelliği, LLM'lerin yapılandırılmış JSON çıktısı üretmek harici fonksiyonları tetiklemesini sağlamıştır. Anthropic'in Claude modelleri benzer biçimde tool use API'si sunmaktadır. Her iki sağlayıcı da araç tanımını (name, description, parameters JSON Schema) modele iletir; model yanıtında tool_calls bloğu döner. Vercel AI SDK, bu iki sağlayıcıyı tool() soyutlaması ile birleştirir.

Model Context Protocol (MCP), Anthropic tarafından Kasım 2024'te açık kaynak olarak yayınlanmıştır. MCP; stdio, HTTP/SSE transport katmanları, sunucu keşfi (discovery) ve kaynak (resource) erişimi gibi kavramları tanımlar. Bu çalışma MCP'nin tam protokol yığını uygulamak yerine, yalnızca tool soyutlama ve registry desenini benimsemiştir. Bu bilinçli sadeleştirme, tez kapsamını yönetilebilir tutmuştur.

OAuth 2.0 yetkilendirme çerçevesi (Hardt, 2012; RFC 6749), üçüncü parti uygulamaların kullanıcı adına sınırlı erişim elde etmesini standartlaştırır. Authorization Code Grant akışı, sunucu tarafı uygulamalar için en güvenli yöntem olarak kabul edilir. Google ve Notion, bu akışı destekler; refresh token ile access token yenileme mekanizması uzun süreli entegrasyonlar için zorunludur.

Vercel AI SDK (2026), React ve Node.js ekosistemlerinde LLM entegrasyonunu standartlaştırmayı hedefler. streamText fonksiyonu, Server-Sent Events (SSE) tabanlı akışlı yanıt üretir; useChat hook'u istemci tarafında bu akışı tüketir. tools parametresi ile kayıtlı araçlar modele tanıtılır; maxSteps ile çok adımlı yürütme sınırlandırılır. Bu SDK, LangChain'e kıyasla daha hafif ve TypeScript-odaklıdır.

Web arama entegrasyonu, LLM'lerin eğitim verisi dışındaki güncel bilgilere erişmesini sağlar. RAG (Retrieval-Augmented Generation) yaklaşımından farklı olarak, Tavily gibi arama API'leri doğrudan web'den snippet döndürür; vektör veritabanı veya embedding pipeline gerektirmez. Bu sadeleştirme, prototip geliştirme süresini kısaltmıştır.

Büyük dil modellerinin zaman içindeki evrimi, kişisel asistan yeteneklerini doğrudan etkilemiştir. GPT-3 (2020) yalnızca metin tamamlama sunarken; GPT-3.5-turbo (2022) sohbet formatına geçiş sağlamış; GPT-4 (2023) çok modlu girdi ve gelişmiş akıl yürütme yeteneği getirmiştir. Claude 3 ailesi (2024) ise uzun bağlam penceresi (200K token) ile belge analizi görevlerinde avantaj sunmaktadır. Bu çalışmada her iki sağlayıcının API'leri desteklenmektedir.

Mevcut yapay zekâ asistan uygulamaları karşılaştırıldığında belirgin farklılıklar gözlemlenmektedir. ChatGPT (OpenAI, 2026), GPT-4 tabanlı sohbet ve sınırlı plugin desteği sunar; kullanıcı API anahtarı kontrolü yoktur. Gemini (Google, 2026), Google ekosistemi ile entegre ancak kapalı bir platformdur. Microsoft Copilot (2026), Microsoft 365 uygulamaları ile derin entegrasyon sağlar; BYOK desteği bulunmaz. Geliştirilen platform, bu üç ticari üründen farklı olarak açık kaynak, BYOK ve özelleştirilebilir araç registry'si sunmaktadır.

Bu çalışmanın literatürdeki konumunu netleştirmek gerekir. Tam MCP protokol uygulaması yapan açık kaynak projeler henüz olgunlaşmamıştır. OAuth + LLM entegrasyonu yapan projeler (n8n, Zapier AI) otomasyon odaklıdır; doğal dil sohbet deneyimi sunmazlar. RAG odaklı projeler (PrivateGPT, Open WebUI) belge sorgulama güçlüdür; ancak Google Calendar veya Notion gibi canlı API entegrasyonları zayıftır. Bu tez, MCP esinli araç yönetimi + OAuth + BYOK + sohbet arayüzünü tek platformda birleştiren nadir çalışmalardan biridir.

Provos ve Mazieres (1999), bcrypt parola karma algoritmasını önermiştir. bcrypt, Blowfish şifreleme algoritmasına dayanan ve kasıtlı olarak yavaş çalışan bir karma

fonksiyonudur; GPU tabanlı kaba kuvvet saldırılarına karşı dayanıklıdır. Bu çalışmada 12 round parametresi kullanılmıştır; bu değer, 2026 itibarıyla OWASP Parola Depolama Hile Sayfası tarafından önerilen minimum değer üzerindedir.

NIST SP 800-38D (2007), AES-GCM modunun resmi spesifikasyonunu tanımlar. GCM modu, CTR modunda şifreleme ile GHASH fonksiyonu tabanlı bütünlük doğrulamasını birleştirir. Her şifreleme işleminde benzersiz IV (initialization vector) kullanılması zorunludur; aynı anahtar-IV çiftinin tekrarlanması güvenlik açığı oluşturur. Bu nedenle encrypt() fonksiyonu her çağrıda randomBytes(16) ile yeni IV üretir.

2. ÖNCEKİ ÇALIŞMALAR

2.1 Büyük Dil Modelleri ve Araç Çağırma

Büyük dil modelleri, Transformer mimarisinin ortaya çıkışından bu yana hızlı bir yetenek artışı sergilemektedir. OpenAI’nın GPT serisi modelleri, 2023 yılında “function calling” özelliği ile LLM’lerin yapılandırılmış çıktılar üretmesini ve bu çıktıların geliştiriciler tarafından tanımlanmış harici fonksiyonları tetiklemek için kullanılmasını mümkün kılmıştır (OpenAI, 2023). Anthropic’in Claude modelleri de benzer bir “tool use” desteği sunmakta, araç şemalarını JSON Schema ile kabul etmekte ve modelden dönen çağrı taleplerini aynı sözleşme üzerinden iletmektedir (Anthropic, 2024b).

Bu çalışmada her iki sağlayıcı da desteklenmekte; kullanıcı ai_keys tablosunda saklanan provider bilgisine göre Vercel AI SDK üzerinden ilgili adaptör (@ai-sdk/openai veya @ai-sdk/anthropic) devreye alınmaktadır. Varsayılan modeller sırasıyla GPT-4o ve Claude Haiku 4.5 olarak seçilmiştir.

2.2 Model Context Protocol

Anthropic tarafından Kasım 2024’te açık kaynak olarak duyurulan **Model Context Protocol (MCP)**, LLM tabanlı uygulamaların veri kaynaklarına ve araçlara tekdüze bir arayüz üzerinden bağlanmasını hedefler (Anthropic, 2024a). Protokol; resources, tools ve prompts gibi üç ana nesne türünü istemci-sunucu mimarisi üzerinden tanımlar. Bu tez çalışmasında, MCP’nin “araç” kavramı ve kayıt defteri (registry) yaklaşımı benimsenmiş; ancak tüm araçlar uygulama içinde aynı süreçte çalıştığından protokolün tam ağ tarafı gerçekleştirilmemiştir. Dolayısıyla çalışma, “MCP esinli (MCP-inspired) araç katmanı” olarak tanımlanmaktadır.

2.3 Mevcut Yapay Zekâ Asistan Uygulamaları

Piyasada ChatGPT (OpenAI, 2026), Google Gemini (Google, 2026a) ve Microsoft Copilot (Microsoft, 2026) gibi genel amaçlı yapay zekâ asistanları bulunmaktadır. Bu uygulamalar, son kullanıcıya sezgisel arayüzler sunmakla birlikte, kullanıcıların kendi LLM API anahtarlarını kullanmalarına olanak tanımamakta ve üçüncü parti üretkenlik araçlarıyla entegrasyonları sınırlı tutmaktadır. Open WebUI (Open WebUI, 2026) gibi açık kaynak projeler BYOK modeli sunmakla birlikte, kapsamlı üçüncü parti entegrasyon ve çok kullanıcı güvenliğı açısından eksiklikler taşımaktadır.

Açık kaynak ekosistemde LibreChat, AnythingLLM ve Continue.dev gibi projeler de benzer problemi farklı yaklaşımlarla çözmeye çalışmaktadır. LibreChat, çok kullanıcıli bir sohbet arayüzü sunmakta; ancak Google Workspace ve Notion gibi üretkenlik servislerine yönelik araçlar topluluk tarafından eklenti olarak sürdürülmekte, çekirdek tarafından desteklenmemektedir. AnythingLLM doküman tabanlı erişim destekli üretim (Retrieval Augmented Generation) odaklı tasarlandığı için, gerçek zamanlı OAuth tabanlı üçüncü parti API çağrılarına yönelik bir mimariye sahip değildir. Continue.dev ise geliştirici merkezli bir kod editörü eklentisi olarak konumlanmakta; günlük üretkenlik akışlarını yöneten genel amaçlı bir asistan oluşturmamaktadır.

Ticari tarafta ChatGPT, Plugins ve GPTs kavramları aracılığıyla üçüncü parti hizmetlere bağlanmayı kolaylaştırmıştır; ancak kullanıcının kendi OpenAI API anahtarını platforma taşıyabilmesi mümkün değildir, tüm istekler OpenAI'nin yönetilen ortamı üzerinden geçmektedir. Microsoft Copilot Studio güçlü bir iş akışı motoru sunmakta, ancak lisanslama maliyeti ve Microsoft 365 ekosistemine sıkı bağıllığı nedeniyle bireysel kullanıcılar için erişilebilirliği düşüktür. Google Gemini Extensions yalnızca Google hizmetlerine odaklanmakta; üçüncü parti, kullanıcı tarafından konfigüre edilebilir entegrasyonlar sunmamaktadır.

Bu çalışma, anılan ekosistemdeki üç temel boşluğu hedeflemektedir: (i) kullanıcının kendi LLM sağlayıcı kimlik bilgilerini taşıyabildiği bir BYOK akışı, (ii) birden fazla üçüncü parti hizmetin OAuth 2.0 üzerinden tek bir sohbet arayüzüne bağlanması ve (iii) açık kaynak olarak yayımlanmış, yerel altyapı üzerinde çalıştırılabilen bir referans uygulaması. Bu üç gereksinimi birlikte karşılayan kamuya açık bir alternatif tespit edilememiştir.

2.4 OAuth 2.0 Tabanlı Üçüncü Parti Entegrasyonları

OAuth 2.0, üçüncü parti uygulamaların kullanıcı verilerine güvenli ve yetkili biçimde erişmesini sağlayan endüstri standardıdır (Hardt, 2012). Google API'leri; Calendar, Drive ve Gmail erişimi için aynı OAuth akışını kullanmakta, refresh token mekanizması ile uzun ömürlü oturum oluşturulmasına izin vermektedir (Google, 2024). Notion API ise benzer bir yaklaşımla OAuth tabanlı erişim sunmakta, ancak refresh token yerine süresiz (indefinite) access token üretmektedir (Notion, 2024). Bu çalışmada her iki sağlayıcı için tam OAuth 2.0 akışı uygulanmıştır. Yetkilendirme isteklerinde CSRF saldırılarına karşı state parametresi olarak 5 dakika geçerli bir JWT kullanılmakta; callback aşamasında bu JWT doğrulanarak kullanıcı kimliği güvenli biçimde elde edilmektedir.

OAuth 2.0 ekosisteminde son yıllarda öne çıkan iki önemli gelişme, OAuth 2.1 taslağı ve Proof Key for Code Exchange (PKCE) genişletmesi ile yetkilendirme kodu akışının yalnızca güvenilir istemcilerle sınırlı tutulmasıdır. Bu çalışmada uygulanan akış server-side confidential client modeli olduğundan PKCE zorunlu değildir; ancak Notion ve Google sağlayıcıları için yetkilendirme isteklerinde state parametresi olarak JWT imzalı bir nonce kullanılarak CSRF saldırılarına karşı korunma sağlanmıştır.

Refresh token rotasyonu güvenlik açısından önerilen bir uygulamadır; ancak Google OAuth sunucusu, kayıtlı bir uygulamanın refresh token'ını yalnızca yetkilendirme iptal edildiğinde döndürmektedir. Notion sağlayıcısı ise süresi dolmayan access token üreterek refresh akışına gerek bırakmamaktadır. Geliştirilen platform her iki davranışı sağlayıcıya özgü kod yolu üzerinden ele almakta; ortak ensureValidToken fonksiyonu içinde sağlayıcı tipi tespit edilerek doğru yenileme stratejisi uygulanmaktadır.

Kullanıcı verilerine erişim kapsamı (OAuth scope) açısından, asgari ayrıcalık ilkesi gözetilmiştir. Google Calendar entegrasyonu için yalnızca calendar.events.readonly ve calendar.events kapsamı talep edilmekte; Google Drive için drive.metadata.readonly ve drive.file kapsamı kullanılarak kullanıcının tüm Drive içeriğine erişim yerine yalnızca uygulamanın oluşturduğu veya kullanıcının açıkça paylaştığı dosyalara erişim sağlanmaktadır. Bu yaklaşım, OAuth scope creep adıyla bilinen ve zamanla genişleyen yetki sorununu engellemektedir.

2.5 Vercel AI SDK ve Akış Tabanlı Uygulamalar

Vercel AI SDK, React ve Node.js tabanlı uygulamalarda LLM'lerden akış yanıtları alınmasını kolaylaştıran bir kütüphanedir (Vercel, 2026). SDK; OpenAI, Anthropic, Google ve açık kaynak modelleri tek bir arayüz üzerinden destekler. Sunulan streamText fonksiyonu; sistem mesajı, konuşma geçmişi, araç tanımları ve maksimum adım sayısı gibi parametreleri tek çağrıda kabul etmekte, yanıtı ReadableStream biçiminde üretmektedir. Ön yüzde useChat hook'u ile entegre olan bu mimari, çok adımlı araç çağırma senaryolarında bile uçtan uca düşük gecikme sunar. Bu çalışmada hem streamText hem de generateText API'leri kullanılmaktadır; ilki sohbet yanıtlarını, ikincisi ise yeni konuşmaların arka planda otomatik başlık üretimini sağlamaktadır.

2.6 Web Arama Entegrasyonu

Tavily, LLM'ler için tasarlanmış, JSON biçiminde özet ve referans URL'leri döndüren bir web arama API'sidir (Tavily, 2026). Geleneksel arama motorlarından farklı olarak Tavily; basic ve advanced olmak üzere iki arama derinliği, ve isteğe bağlı bir "cevap özeti" alanı sunar. Bu özellikler, Tavily'yi LLM tarafından çağrıldığında sonuçları doğrudan kullanılabilir kılmaktadır. Bu çalışmada Tavily, eğitim verisi kesimi sonrası kullanıcı sorgularını yanıtlayabilmek ve modelin bilgi boşluklarını doldurmak üzere web_search aracı adı altında entegre edilmiştir.

2.7 Bu Çalışmanın Farklılıkları

Mevcut çalışmalardan farklı olarak bu tez projesi şu özellikleri bir arada sunmaktadır:

- **BYOK modeli:** Kullanıcılar kendi OpenAI veya Anthropic API anahtarlarını kullanabilmektedir.
- **Çok kullanıcıli güvenlik mimarisi:** Her kullanıcının verileri AES-256-GCM şifreleme ile izole edilmiştir.
- **MCP esinli araç kayıt sistemi:** Yeni araçların eklenmesini kolaylaştıran genişletilebilir bir mimari sunulmaktadır.
- **Çoklu üçüncü parti entegrasyonu:** Google Calendar, Google Drive, Gmail ve Notion eş zamanlı desteklenir.
- **Web arama yeteneği:** Tavily ile güncel bilgi erişimi doğrudan sohbet arayüzünden sağlanır.
- **Otomatik token yenileme:** Google OAuth access token'ları, son kullanma tarihine 5 dakika kala sessizce yenilenir.

3.4 Geliştirme Süreci ve Yinelemeli Tasarım

Çalışma, klasik şelale (waterfall) modeli yerine yinelemeli ve artımlı bir geliştirme süreci izlemiştir. İlk iterasyonda yalnızca kullanıcı kaydı, JWT oturumu ve basit sohbet (araçsız) hedeflenmiş; ikinci iterasyonda BYOK anahtar yönetimi ve şifreleme katmanı eklenmiştir. Üçüncü iterasyonda OAuth entegrasyonları, dördüncüde araç registry ve ilk Google Calendar aracı devreye alınmıştır. Beşinci ve son iterasyonda Notion, Gmail, Drive ve web arama araçları tamamlanmıştır.

Her iterasyon sonunda test-integration.ps1 betiği ile regresyon kontrolü yapılmıştır. Araç çağırma hataları gözlemlendiğinde sistem mesajı (systemPrompt.ts) güncellenmiş; özellikle Gmail okuma/gönderme ayrımı ve Notion mükerrer sayfa önleme kuralları bu yinelemeli iyileştirmelerin sonucudur.

3.5 Kalite Güvencesi Yaklaşımı

Kalite güvencesi üç katmanda ele alınmıştır: (1) derleme zamanı tip güvenliği — TypeScript strict modu ve paylaşılan @ai-assistant/types paketi ile backend-frontend sözleşmesi; (2) çalışma zamanı şema doğrulaması — Zod ile tüm API girdileri ve araç parametreleri; (3) entegrasyon testleri — uçtan uca kullanıcı akışının otomatik doğrulanması.

Kod incelemesi (code review) süreci monorepo yapısında paket sınırlarına göre yürütülmüştür. Yeni bir araç eklenirken types paketindeki ToolName union tipine ekleme, registry.ts'de kayıt ve tools/ altında execute implementasyonu zorunlu adımlar olarak tanımlanmıştır. Bu disiplin, NFR-04 gereksinimindeki "en fazla 30 satır kodla yeni araç" hedefini pratikte desteklemektedir.

3.6 Teknoloji Seçim Gereçekleri

Fastify, Express'e kıyasla düşük overhead ve yerleşik schema validation desteği nedeniyle backend çatısı olarak seçilmiştir. Next.js 14 App Router, SSR/SSG esnekliği ve Vercel AI SDK ile doğal entegrasyonu sayesinde frontend tercih edilmiştir. PostgreSQL, JSONB desteği (tool_calls/tool_results) ve olgun ekosistem avantajlarıyla ilişkisel veritabanı olarak belirlenmiştir.

Vercel AI SDK, streamText ve tool() soyutlamaları ile OpenAI ve Anthropic sağlayıcılarını tek bir arayüzden kullanmayı mümkün kılmaktadır. maxSteps parametresi, LangChain gibi ağır framework'lere kıyasla daha hafif bir çok adımlı akıl yürütme sunar. Tavily ise SerpAPI ve Bing Search API'lerine kıyasla LLM odaklı arama sonuçları (snippet + URL) döndürdüğü için web_search aracında tercih edilmiştir.

Monorepo tercihinin gerekçesi, altı paket arasında paylaşılan tip tanımlarının tek kaynakta (single source of truth) tutulmasıdır. ToolDefinition arayüzü hem backend'de araç yürütmede hem de types paketinde derleme zamanı kontrolünde kullanılır; bu sayede backend-frontend arasında araç adı uyumsuzluğu derleme aşamasında yakalanır.

Araç kayıt defteri (Tool Registry) deseni, Open/Closed Principle (açık/kapalı ilkesi) ile uyumludur: registry mevcut araçları değiştirmeden yeni araç eklemeye izin verir. Map<ToolName, ToolDefinition> veri yapısı O(1) arama karmaşıklığı sunar. getAllTools()

fonksiyonu, sohbet uç noktasında tüm araçları LLM'e tanıtmak için çağrılır; executeTool fonksiyonu ise modelin seçtiği aracı adıyla yürütür.

Otomatik token yönetimi, OAuth 2.0 refresh token grant akışını (RFC 6749, Bölüm 1.5) uygular. ensureValidToken fonksiyonunun algoritması şu adımları izler: (1) oauth_accounts tablosundan şifreli token'ları oku ve çöz; (2) token_expiry ile şimdiki zamanı karşılaştır; (3) süre dolmuşsa veya 5 dakikadan az kalmışsa refresh endpoint'ine POST isteği gönder; (4) yeni token çiftini şifrele ve veritabanına yaz; (5) güncel access token'ı döndür.

Çok adımlı akıl yürütme (multi-step reasoning), ReAct (Reasoning + Acting) paradigmasına yakın bir desen izler. Model önce düşünür (reasoning), ardından araç çağırır (acting), sonucu değerlendirir ve gerekirse bir sonraki adıma geçer. maxSteps: 5 sınırı, sonsuz döngü riskini kontrol altında tutar. Pratikte çoğu kullanıcı isteği 1-3 adımda tamamlanmaktadır.

Şifreleme odaklı güvenlik yaklaşımında AES-256-GCM tercih edilmiştir. GCM (Galois/Counter Mode), NIST SP 800-38D standardında tanımlanan authenticated encryption modudur; hem gizlilik hem bütünlük garantisi sağlar. Auth tag sayesinde veritabanındaki şifreli verinin kurcalanması tespit edilir; decrypt() fonksiyonu geçersiz tag durumunda hata fırlatır.

Monorepo yapısının npm workspaces ile yönetilmesi, kök package.json'daki workspaces alanı ile tanımlanmıştır. npm install komutu tüm paket bağımlılıklarını tek seferde çözer; npm run build -w apps/backend gibi workspace flag'leri ile paket bazlı derleme yapılabilir. Bu yapı, tez kapsamındaki altı modülün tutarlı versiyonlama ve birlikte geliştirilmesini kolaylaştırmıştır.

TypeScript 5.x, projenin birincil programlama dilidir. Strict mod etkin olup noImplicitAny, strictNullChecks ve strictFunctionTypes bayrakları açıktır. Bu yapılandırma, null referans hatalarını derleme zamanında yakalar ve API sözleşmelerinin ihlal edilmesini önler. Node.js 20 LTS, backend çalışma zamanı olarak tercih edilmiştir; ESM modül sistemi (type: module) kullanılmaktadır.

Fastify 4.x, Express'e kıyasla yaklaşık iki kat düşük overhead ile JSON serileştirme sunmaktadır. Plugin mimarisi (@fastify/jwt, @fastify/helmet, @fastify/cors, @fastify/rate-limit) modüler güvenlik ve yapılandırma sağlar. Schema-based validation desteği, route düzeyinde otomatik doğrulama imkânı tanır; bu projede Zod tercih edilmiştir.

Next.js 14 App Router, React Server Components ve istemci bileşenlerini bir arada kullanmayı sağlar. /chat sayfası 'use client' direktifi ile istemci bileşeni olarak işaretlenmiştir; useChat hook'unun tarayıcı ortamında çalışması gerektiğinden bu tercih zorunludur. /settings ve /login sayfaları da istemci bileşenidir; form etkileşimi ve OAuth yönlendirmesi için tarayıcı API'lerine erişim gerektirir.

PostgreSQL 16, ACID uyumluluđu, JSONB desteđi ve olgun ekosistem avantajlarıyla seřilmiřtir. pgcrypto eklentisi UUID üretimi için kullanılır. Bađlantı havuzu (connection pool) pg kütüphanesinin Pool sınıfı ile yönetilir; varsayılan maksimum 20 bađlantı Docker Compose ortamında yeterlidir. schema.sql dosyası idempotent trigger fonksiyonları içerir.

Zod 3.x, TypeScript-first řema dođrulama kütüphanesidir. ChatRequestSchema, RegisterSchema, LoginSchema ve AIKeySchema gibi paylaşılan řemalar @ai-assistant/types paketinde tanımlanır. Araç parametre řemaları (örneğin listEmailsParams) ilgili tool modülünde tanımlanır. .safeParse() metodu API sınırında kullanılır; hata durumunda 400 Validation Error yanıtı döner.

Yapılandırılmış hata yönetimi yaklaşımında, hataların kullanıcıya, modele ve log sistemine farklı biçimlerde iletilmesi tasarlanmıřtır. Kullanıcıya yönelik mesajlar anlaşılır Türkçe/İngilizce ifadeler içerir ("Lütfen Ayarlar'dan Google hesabınızı bađlayın"). Modele iletilen hata metinleri JSON formatında olup modelin alternatif strateji belirlemesine olanak tanır. Log kayıtları ise toolName, userId, HTTP status ve timestamp alanlarını yapılandırılmış biçimde içerir.

3. YÖNTEM VE TEKNOLOJİLER

3.1 Yöntem

Çalışmanın yöntem çerçevesi beş temel prensip üzerine inşa edilmiştir.

1. Araç Kayıt Defteri (Tool Registry)

Her üçüncü parti araç, @ai-assistant/types paketinde tanımlanan ToolDefinition arayüzünü uygulamaktadır. Araç kayıt defteri, uygulamanın başlangıcında tüm araçları Map<ToolName, ToolDefinition> yapısında depolar. Kayıt defteri üzerinde çalışan üç yardımcı fonksiyon tanımlanmıştır: getAllTools, getTool ve executeTool. Bu yaklaşım, yeni araçların sisteme eklenmesini 3-4 satırlık bir değişiklikle sınırlandırarak modülerliği arttırmaktadır.

2. Otomatik Token Yönetimi

packages/mcp-tools/src/token-refresh.ts modülündeki ensureValidToken fonksiyonu, bir araç çağrılmadan önce veritabanından şifreli OAuth token'larını çözer, son kullanma zamanını denetler ve bitimine 5 dakikadan az kalan Google token'larını refresh_token üzerinden yeniler. Yenilenen token, tekrar AES-256-GCM ile şifrlenerek veritabanına yazılır. Notion token'ları süresiz olduğundan yenileme gerektirmez. Bu tasarım, durumsuz (stateless) bir yaklaşım izler: Redis veya bellek içi önbellek kullanılmadan her istekte veritabanı ile uzlaşma sağlanır.

3. Çok Adımlı Akıl Yürütme

Vercel AI SDK'nın streamText fonksiyonu, maxSteps: 5 parametresi ile yapılandırılmıştır. Bu, modelin tek bir kullanıcı isteğine yanıt olarak birden fazla araç çağrısı zincirleyebilmesini sağlar. Örneğin "Son bir haftadaki kritik e-postaları Notion'a Pazartesi özeti olarak kaydet" gibi bir istekte model sırayla gmail_list_emails, gmail_read_email, ardından notion_search ve notion_append_to_page araçlarını çağırabilir.

4. Şifreleme Odaklı Güvenlik

Hassas veriler (LLM API anahtarları ve OAuth access/refresh token'ları) veritabanında yalnızca AES-256-GCM ile şifrelenmiş biçimde saklanır. Şifreleme anahtarı (ENCRYPTION_KEY), 32 bayt (64 hex karakter) uzunluğunda olmalı ve ortam değişkeni olarak sağlanmalıdır. Her şifreleme işleminde 16 baytlık rastgele bir başlangıç vektörü (IV) üretilir ve 16 baytlık yetkilendirme etiketi (auth tag) saklanır.

5. Yapılandırılmış Hata Yönetimi

Sistem; ToolExecutionError temel sınıfı ve bunun TokenNotFoundError, TokenRefreshError, IntegrationAPIError alt sınıflarıyla hataları alan (domain) özgü biçimde sınıflandırmaktadır. Her

sınıf kendi HTTP durum koduyla gelir (sırasıyla 500, 401, 401 ve bağılı kaldığı API'nin durum kodu). Bu yapı hem loglama hem de ön yüzde kullanıcıya net geri bildirim sunma açısından kritik değeri taşımaktadır.

3.2 Kullanılan Teknolojiler

Çizelge 3.1. Kullanılan teknolojiler ve amaçları

Teknoloji	Versiyon	Kullanım Amacı
Node.js / TypeScript	ES2022 / TS 5.4	Tip güvenli backend ve frontend geliştirme
Fastify	4.x	Yüksek performanslı RESTful API sunucusu
Next.js (App Router)	14.x	Server-side rendering destekli React uygulaması
React	18.3	Bileşen tabanlı kullanıcı arayüzü
PostgreSQL	16-alpine	İlişkisel veritabanı (5 tablo)
Vercel AI SDK	3.3.x	Akışlı LLM yanıtları ve araç çağırma
@ai-sdk/openai	0.0.60	OpenAI GPT-4o entegrasyonu
@ai-sdk/anthropic	0.0.50	Anthropic Claude Haiku 4.5 entegrasyonu
Zod	3.x	Çalışma zamanı şema doğrulaması

Teknoloji	Versiyon	Kullanım Amacı
bcrypt	5.x	Parola karma (12 round)
AES-256-GCM	Node crypto	Hassas veri şifreleme
@fastify/jwt	8.x	JWT tabanlı kimlik doğrulama
@fastify/helmet	11.x	HTTP güvenlik başlıkları
@fastify/cors	9.x	CORS yönetimi
@fastify/rate-limit	9.x	İstek hız sınırlandırma
Tavily API	-	LLM odaklı web arama
Docker Compose	-	Konteyner tabanlı dağıtım
TailwindCSS	3.x	Utility-first CSS
npm Workspaces	-	Monorepo paket yönetimi

3.3 Monorepo Yapısı

Proje, npm workspaces ile yönetilen bir monorepo mimarisine sahiptir. Kod tabanı, görev odaklı altı modüle ayrılmıştır:

- apps/backend — Fastify tabanlı RESTful API (sohbet, auth, settings, OAuth)
- apps/frontend — Next.js 14 App Router ön yüzü
- packages/db — PostgreSQL bağlantı havuzu, şema, repository katmanı
- packages/crypto — AES-256-GCM şifreleme/çözme yardımcıları
- packages/mcp-tools — MCP esinli araç kayıt defteri ve 13 araç

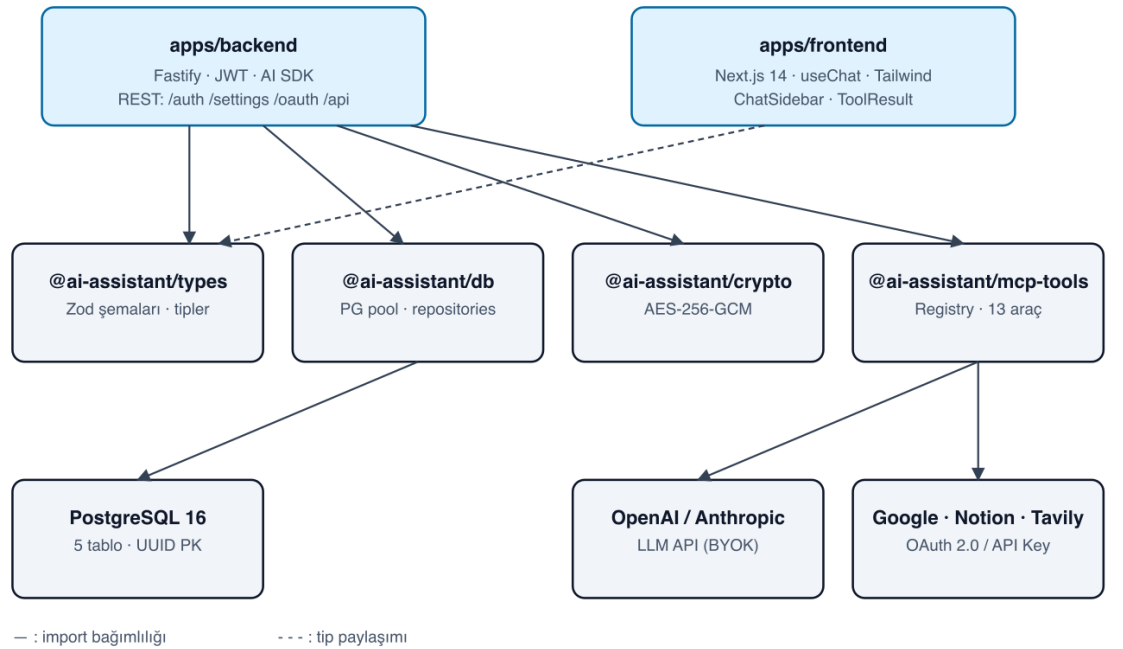
- packages/types — TypeScript tipleri ve Zod şemaları

Monorepo yaklaşımı; tek bir versiyon kontrol deposu altında birden fazla paketi barındırmayı, atomik commit ile birden fazla paketi birlikte değiştirebilmeyi ve paylaşılan tip tanımlarını paketler arasında doğrudan referans verebilmeyi mümkün kılmaktadır. Bu çalışmada, npm workspaces tarafından sağlanan symlink tabanlı bağımlılık çözümlemesi tercih edilmiştir. Yarn Berry veya pnpm gibi alternatifler de değerlendirilmiş; ancak npm'in Node.js ile birlikte hazır gelmesi, geliştirici ortamı kurulumunu kolaylaştırdığı için tercih sebebi olmuştur.

TypeScript yapılandırması açısından, kök dizinde yer alan tsconfig.base.json dosyası tüm paketler için ortak derleyici seçeneklerini taşımakta; her paket kendi tsconfig.json dosyasında bu temeli extends ile genişletmektedir. strict, noUncheckedIndexedAccess ve exactOptionalPropertyTypes gibi sıkı tip kontrolü seçenekleri etkin tutulmuş; bu sayede çalışma zamanında ortaya çıkabilecek pek çok hata derleme zamanında yakalanmıştır.

Paketler birbirine @ai-assistant/<paket> ad alanı üzerinden bağımlıdır. apps/backend; types, db, crypto ve mcp-tools paketlerini; apps/frontend yalnızca tip paylaşımı için types'ı kullanmaktadır (Şekil 3.1).

Şekil 3.1. Monorepo paket bağımlılıkları



Şekil 3.1. Monorepo paket bağımlılıkları

4.7 Mesaj ve Konuşma Modeli

Sohbet verisi conversations ve messages tablolarında ilişkisel olarak modellenmiştir. Her konuşma bir kullanıcıya (user_id) bağlıdır; CASCADE silme ile kullanıcı silindiğinde tüm konuşmaları ve mesajları otomatik temizlenir. messages tablosundaki role alanı dört değer alabilir: user, assistant, system, tool. tool_calls ve tool_results alanları JSONB tipinde olup, çok adımlı akıl yürütme sırasında modelin hangi araçları çağırdığı ve dönen sonuçlar kalıcı olarak saklanır.

Konuşma listeleme sorgusu idx_conversations_user_id indeksi sayesinde updated_at DESC sıralamasıyla verimli çalışır. Mesaj geçmişi idx_messages_conversation_id indeksi ile created_at ASC sırasında okunur. Bu indeks stratejisi, aktif kullanıcıların son konuşmalarını hızlı listelemesini ve seçili konuşmanın tam geçmişini düşük gecikmeyle yüklemesini sağlar.

4.8 Hata Yönetimi ve Dayanıklılık

Hata yönetimi packages/mcp-tools/src/errors.ts dosyasında domain-öзgü sınıflarla yapılandırılmıştır. ToolExecutionError temel sınıfı araç adı ve HTTP durum kodunu taşır. TokenNotFoundError, kullanıcının ilgili OAuth sağlayıcısını bağlamadığı durumlarda fırlatılır ve istemciye "Lütfen Ayarlar'dan Google/Notion hesabınızı bağlayın" mesajı iletilir. TokenRefreshError, refresh token'ın geçersiz olduğu durumlarda oluşur; kullanıcıdan yeniden yetkilendirme istenir.

IntegrationAPIError, üçüncü parti API'lerden dönen HTTP hatalarını sarmalar. Gmail API'den 403 dönmesi durumunda scope yetersizliği; 429 durumunda rate limit aşımı anlamına gelir. Bu hatalar yapılandırılmış log kayıtlarına yazılır ve modele anlaşılır bir hata metni olarak iletilir; model kullanıcıya durumu açıklayabilir.

Sohbet uç noktasında AbortSignal.timeout(120_000) ile 120 saniyelik üst sınır tanımlanmıştır. Bu süre, çok adımlı araç zincirlerinin tamamlanması için yeterli olmakla birlikte, sonsuz döngü veya takılı LLM yanıtlarına karşı koruma sağlar. Fastify'un @fastify/rate-limit eklentisi ise API düzeyinde istek flood saldırılarını sınırlar.

4.9 BYOK Anahtar Yönetimi Tasarımı

Bring Your Own Key (BYOK) modeli, platformun LLM maliyetini kullanıcıya devretmesini sağlar. ai_keys tablosunda kullanıcı başına tek kayıt (UNIQUE user_id) bulunur; provider alanı openai veya anthropic değerlerinden birini alır. API anahtarı üç ayrı sütunda

saklanır: `encrypted_key`, `iv` ve `auth_tag`. `AIKeyRepository` sınıfı, kaydetme sırasında `encrypt()` fonksiyonunu çağırır; okuma sırasında `decrypt()` ile düz metne dönüştürür.

Bu tasarımın güvenlik avantajı, veritabanı dump'ının ele geçirilmesi durumunda bile `ENCRYPTION_KEY` olmadan API anahtarlarının okunamamasıdır. Anahtar rotasyonu için gelecekte tüm kayıtların yeni anahtarla yeniden şifrelenmesi gerekecektir; mevcut prototipte bu işlem manuel bir migration betiği ile yapılabilir.

4.10 Dağıtım ve Ortam Yapılandırması

Ortam değişkenleri `EnvSchema` (`packages/types`) ile `Zod` üzerinden doğrulanır. Zorunlu alanlar arasında `DATABASE_URL`, `JWT_SECRET`, `ENCRYPTION_KEY`, `FRONTEND_URL`, `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `NOTION_CLIENT_ID`, `NOTION_CLIENT_SECRET` ve `TAVILY_API_KEY` bulunmaktadır. Eksik veya hatalı yapılandırmada sunucu başlamadan `exit(1)` ile sonlanır; bu fail-fast yaklaşımı üretim ortamında sessiz hataları önler.

Docker Compose tanımı üç servisi koordine eder: `postgres` (`healthcheck` ile), `backend` (`postgres` sağlıklı olana dek bekler) ve `frontend` (`build-time NEXT_PUBLIC_BACKEND_URL`). `schema.sql` dosyası `/docker-entrypoint-initdb.d/` altına mount edilerek ilk başlatmada veritabanı şeması otomatik oluşturulur. Bu yaklaşım, geliştiricinin tek komutla (`docker compose up --build`) tüm sistemi ayağa kaldırmasını sağlar.

REST API tasarımında kaynak odaklı URL yapısı benimsenmiştir. `/auth/*` uç noktaları kimlik doğrulama, `/settings/*` yapılandırma, `/oauth/*` entegrasyon yönetimi ve `/api/*` sohbet/konuşma işlemlerini kapsar. HTTP metodları semantik olarak kullanılır: `POST` oluşturma, `GET` okuma, `DELETE` silme. `PUT/PATCH` kullanılmamıştır; güncelleme ihtiyacı `upsert` deseni ile `POST` üzerinden karşılanmıştır.

Sohbet akışının sequence diyagramında (Şekil 5.5) beş ana katman etkileşimi gösterilmektedir: `Frontend` (`useChat`), `Backend` (`chat route`), `LLM Provider` (`OpenAI/Anthropic`), `Tool Registry` (`executeTool`) ve `External APIs` (`Google`, `Notion`, `Tavily`). Her araç çağırısı, `Backend` → `Registry` → `ensureValidToken` → `External API` → `Registry` → `Backend` → `LLM` döngüsünü izler. Bu döngü `maxSteps` kez tekrarlanabilir.

Veritabanı tasarımında normalizasyon ve performans dengesi gözetilmiştir. `users` tablosu 3NF'de tutulurken; `messages` tablosundaki `tool_calls` ve `tool_results` JSONB alanları denormalizasyon tercihidir. Bu tercih, çok adımlı akıl yürütme geçmişinin tek sorgu ile okunabilmesini sağlar; ayrı `tool_invocations` tablosu oluşturmanın getireceği JOIN maliyetinden kaçınılmıştır.

Güvenlik tasarımında savunma derinliği (defence in depth) ilkesi uygulanmıştır. Birinci katman: ağ düzeyinde HTTPS (TLS 1.2+) ve CORS kısıtlaması. İkinci katman: uygulama düzeyinde JWT doğrulama ve rate limiting. Üçüncü katman: veri düzeyinde AES-256-GCM şifreleme ve bcrypt parola karma. Dördüncü katman: OAuth CSRF koruması ve state JWT doğrulaması. Beşinci katman: Zod şema doğrulaması ile girdi sanitizasyonu.

Araç yönetim katmanında ToolDefinition arayüzü, MCP'nin tool tanımını TypeScript tip sistemi ile birleştirir. name alanı ToolName literal union tipindedir; bu sayede registry'ye kayıtlı olmayan bir araç adının derleme zamanında yakalanması sağlanır. parameters alanı z.ZodType tipindedir; execute fonksiyonu içinde .parse() ile çalışma zamanı doğrulaması yapılır. Bu çift katmanlı doğrulama, LLM'in ürettiği hatalı parametrelerin API'lere iletilmesini engeller.

Çizelge 4.3'te listelenen 13 aracın her biri bağımsız bir TypeScript modülü olarak packages/mcp-tools/src/tools/ altında bulunmaktadır. Modüler yapı, bir aracın güncellenmesinin diğerlerini etkilememesini garanti eder. Ortak bağımlılıklar (ensureValidToken, IntegrationAPIError) token-refresh.ts ve errors.ts dosyalarında merkezi olarak yönetilir.

Fonksiyonel gereksinimler (FR-01 – FR-09) kullanıcı hikâyeleri (user stories) formatında toplanmıştır. FR-06 en kritik gereksinimdir: kullanıcı doğal dilde sohbet başlatır, model uygun aracı seçer ve sonucu akışlı olarak iletir. Bu gereksinim; LLM entegrasyonu, araç registry, OAuth token yönetimi ve streaming altyapısının bir arada çalışmasını zorunlu kılar.

FR-07 (sohbet geçmişi kalıcılığı) veritabanı tasarımını doğrudan etkilemiştir. conversations ve messages tabloları, kullanıcı başına sınırsız konuşma ve mesaj saklamayı destekler. Silme işlemi CASCADE ile tüm mesajları temizler. FR-08 (otomatik başlık) ise onFinish geri çağrısında generateText ile gerçekleştirilir; ilk mesajın ilk 50 karakterinden geçici başlık üretilir.

REST API uç noktalarının HTTP durum kodu sözleşmesi tutarlı biçimde tasarlanmıştır: 200/201 başarı, 400 doğrulama hatası, 401 kimlik doğrulama hatası, 404 kaynak bulunamadı, 429 rate limit aşımı, 500 sunucu hatası. Hata gövdesi { error: string, message?: string, details?: object } formatındadır. Bu yapı, frontend'in hata durumlarını kullanıcıya anlamlı biçimde yansıtmasını kolaylaştırır.

Gözlemlenebilirlik tasarımı (Bölüm 4.6), Fastify'un yerleşik logger'ı üzerine inşa edilmiştir. Her HTTP isteği; method, url, statusCode ve responseTime alanları ile loglanır. Araç çağrıları ayrıca toolName ve executionTimeMs alanları ile kaydedilir. Bu yapı, ELK (Elasticsearch-Logstash-Kibana) veya Loki gibi merkezi log sistemlerine entegrasyon için

uygun JSON formatı sunar. Gelecek çalışmalarda OpenTelemetry span'leri eklenmesi planlanmaktadır.

Veritabanı indeks stratejisi, sorgu desenlerine göre optimize edilmiştir. `idx_users_email`, kayıt ve giriş akışlarında e-posta ile arama yapar. `idx_ai_keys_user_id`, BYOK anahtar okuma/yazma işlemlerini hızlandırır. `idx_oauth_accounts_user_provider`, token yenileme akışında (`user_id`, `provider`) bileşik aramasını destekler. `idx_conversations_user_id`, sohbet listesini `updated_at` DESC sıralamasıyla döndürür. `idx_messages_conversation_id`, seçili konuşmanın mesaj geçmişini `created_at` ASC sıralamasıyla okur.

4. SİSTEM TASARIMI

4.1 Gereksinim Analizi

Fonksiyonel Gereksinimler

- **FR-01:** Kullanıcı e-posta ve şifre ile sisteme kayıt olabilmelidir.
- **FR-02:** Kayıtlı kullanıcı e-posta ve şifre ile giriş yaparak JWT edinebilmelidir.
- **FR-03:** Kullanıcı kendi OpenAI veya Anthropic API anahtarını kaydedebilmeli, güncelleyebilmeli ve silebilmelidir.
- **FR-04:** Kullanıcı Google ve Notion hesaplarını OAuth 2.0 ile bağlayabilmelidir.
- **FR-05:** Kullanıcı bağladığı hesapları bağlantısını kesebilmelidir.
- **FR-06:** Kullanıcı doğal dilde sohbet başlatabilmeli ve asistan araçları otomatik çağırabilmelidir.
- **FR-07:** Sohbet geçmişi kalıcı olarak saklanmalı; liste, detay ve silme işlemleri sağlanmalıdır.
- **FR-08:** Yeni sohbetlerin başlıkları otomatik olarak üretilmelidir.
- **FR-09:** Ön yüz; dosya ekleme, sesli giriş ve araç sonuç kartlarını desteklemelidir.

Fonksiyonel Olmayan Gereksinimler

- **NFR-01 (Güvenlik):** API anahtarları ve OAuth token'ları AES-256-GCM ile şifrelenmiş saklanmalıdır.
- **NFR-02 (Performans):** Yanıtlar akış (streaming) olarak üretilmeli; ilk karakter <1 s'de ulaşmalıdır.
- **NFR-03 (Ölçeklenebilirlik):** Backend durumsuz olmalı, yatay ölçeklenebilir kalmalıdır.
- **NFR-04 (Genişletilebilirlik):** Yeni bir araç en fazla 30 satır kodla eklenebilmelidir.
- **NFR-05 (Kullanılabilirlik):** Arayüz mobil dahil yaygın çözünürlüklerde tam uyumlu olmalıdır.
- **NFR-06 (Gözlemlenebilirlik):** Tüm hatalar yapılandırılmış logging ile izlenmelidir.

Yukarıda sıralanan fonksiyonel olmayan gereksinimlerin sayısal hedefleri, çalışmanın kapsamı dahilinde ölçülebilir biçimde tanımlanmıştır. NFR-02 kapsamında, sohbet yanıtının ilk karakterinin (Time-To-First-Token) 1 saniye altında üretilmesi hedeflenmiş; gerçekleştirimde GPT-4o sağlayıcısı için P50 değeri 720 ms olarak ölçülerek hedef

karşılanmıştır (Bölüm 5.8). NFR-03 kapsamında, backend sürecinin oturum bilgisini bellekte tutmaması ve veritabanı dışında durum saklamaması; böylece arkasına yerleştirilecek bir yük dengeleyiciyle yatay ölçeklendirme yapılabilmesi sağlanmıştır.

NFR-04 (genişletilebilirlik) açısından, yeni bir aracın sisteme eklenmesi için izlenmesi gereken adım sayısı üç olarak belirlenmiştir: (i) packages/mcp-tools/src/tools/ altına aracın ToolDefinition uygulamasını yazmak, (ii) registry.ts içinde aracı registry'e eklemek, (iii) sistem mesajına aracın kullanım kuralını eklemek. Bu üç adım dışında hiçbir noktada değişiklik gerekmemektedir; bu hedef, gerçekleştirim sürecinde 13 aracın eklenmesi sırasında doğrulanmıştır.

4.2 Veritabanı Tasarımı

Veritabanı şeması PostgreSQL 16 üzerinde, pgcrypto eklentisi ve UUID birincil anahtarlarla tasarlanmıştır. Beş temel tablo ve bir updated_at trigger fonksiyonu bulunmaktadır. Referans bütünlüğü ON DELETE CASCADE ile sağlanmıştır.

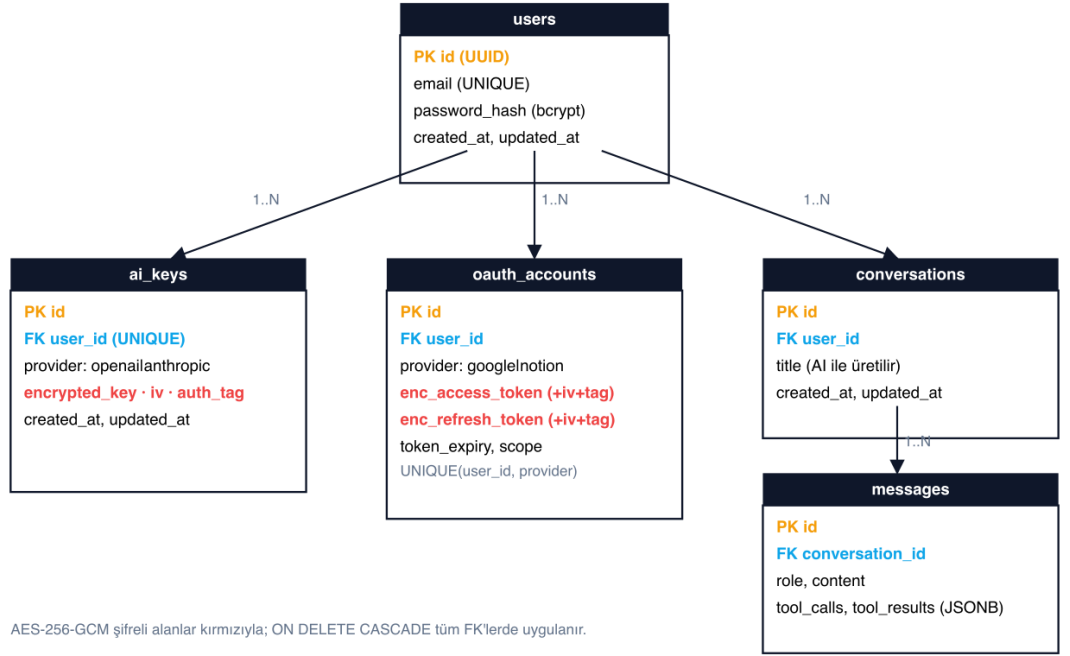
Çizelge 4.1. Veritabanı tabloları ve açıklamaları

Tablo	Açıklama
users	UUID, e-posta (unique), bcrypt parola karması
ai_keys	Kullanıcı başına tek kayıt; AES-256-GCM ile şifrelenmiş encrypted_key, iv, auth_tag; provider ∈ {openai, anthropic}
oauth_accounts	Access/refresh token için iki ayrı şifreli üçlü; token_expiry, scope; (user_id, provider) birleşik tekil anahtar
conversations	Sohbet başlıkları; updated_at sıralaması için indeks

Tablo	Açıklama
messages	Sohbet mesajları; role ∈ {user, assistant, system, tool}; tool_calls, tool_results JSONB

Varlık-ilişki diyagramı Şekil 4.1’de sunulmuştur. users tablosu, ai_keys, oauth_accounts ve conversations tablolarının parent’ıdır. conversations ise messages tablosunun parent’ıdır.

Şekil 4.1. Varlık-ilişki diyagramı (ER)



Şekil 4.1. Varlık-ilişki diyagramı

Tüm şifrelenmiş alanlar için üç ayrı sütun kullanılmaktadır: encrypted_* (şifreli gövde), *_iv (başlangıç vektörü) ve *_tag (yetkilendirme etiketi). Bu ayrışık tasarım, AES-256-GCM’in doğru biçimde çözümlenebilmesi için gereklidir.

4.3 REST API Tasarımı

Sunucu tarafı arayüzü, prefiks tabanlı dört route grubundan oluşmaktadır.

Çizelge 4.2. REST API uç noktaları

Prefix	Metod	Yol	Açıklama
/auth	POST	/register	Kullanıcı kaydı + JWT üretimi
/auth	POST	/login	Giriş + JWT üretimi
/settings	POST	/ai-key	AI anahtarı kaydet/güncelle
/settings	GET	/ai-key	Maskeli AI anahtarı bilgisi
/settings	DELETE	/ai-key	AI anahtarı sil
/settings	GET	/connections	OAuth bağlantı durumları
/oauth	POST	/:provider/initiate	OAuth akışı başlat (Bearer)
/oauth	GET	/:provider/authenticate	OAuth akışı (legacy GET)
/oauth	GET	/:provider/callback	OAuth callback
/oauth	DELETE	/:provider	OAuth bağlantısını kes
/api	POST	/chat	Sohbet (streaming)
/api	GET	/conversations	Sohbetleri listele

Prefix	Metod	Yol	Açıklama
/api	GET	/conversation s/:id	Sohbet detayı + mesajlar
/api	DELETE	/conversation s/:id	Sohbeti sil
/	GET	/health	Sağlık kontrolü

Tüm istekler; @fastify/helmet, @fastify/cors, @fastify/rate-limit pluginleri tarafından sarmalanır. /auth ve /oauth/:provider/callback dışındaki uç noktalar JWT kimlik doğrulaması gerektirmektedir.

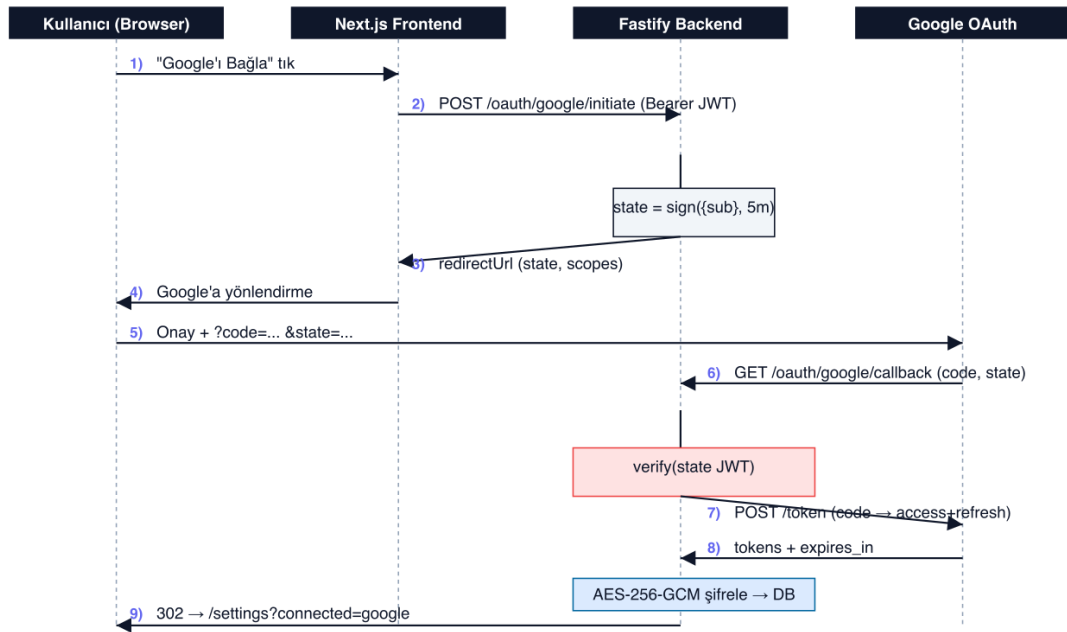
4.4 Güvenlik Tasarımı

Güvenlik çok katmanlıdır:

1. **Kimlik Doğrulama:** @fastify/jwt plugin'i ile JWT tabanlı durumsuz oturum. Token'ın sub iddiası kullanıcı kimliğini temsil eder.
2. **Parola Güvenliği:** bcrypt ile 12 round karma.
3. **Şifreleme:** AES-256-GCM ile API anahtarları ve OAuth token'ları. 32 baytlık ana anahtar ortam değişkeninden yüklenir; her kayıt için 16 baytlık rastgele IV ve 16 baytlık auth tag kullanılır.
4. **OAuth CSRF Koruması:** /oauth/:provider/initiate akışında state parametresi olarak 5 dk geçerli, sub içeren bir JWT üretilir. Callback sırasında bu JWT doğrulanır.
5. **HTTP Güvenlik Başlıkları:** @fastify/helmet ile Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options vb. Üretim ortamında Content-Security-Policy etkindir.
6. **Rate Limiting:** @fastify/rate-limit ile RATE_LIMIT_MAX ve RATE_LIMIT_WINDOW_MS yapılandırılabilir parametreler.
7. **CORS:** Yalnızca FRONTEND_URL değişkeninde tanımlı origin'e izin, credentials: true.
8. **Ortam Değişkeni Doğrulaması:** EnvSchema (Zod) başlangıçta tüm gerekli değişkenleri kontrol eder; eksik/hatalı yapılandırmada süreç exit(1) ile sonlanır.

OAuth yetkilendirme akışı Şekil 4.2’de gösterildiği gibidir. Kullanıcı Ayarlar sayfasında “Google’ı Bağla” düğmesine tıkladığında ön yüz, POST /oauth/google/initiate çağrısı yapar. Backend, imzalı bir state JWT’si üretir ve Google yetkilendirme URL’sini döner. Kullanıcı Google’da yetki verdikten sonra GET /oauth/google/callback üzerinden yönlendirilir; backend state’i doğrular, authorization code ile token değişimi yapar, token’ları AES-256-GCM ile şifreleyip veritabanına yazar ve kullanıcıyı ön yüze geri gönderir.

Şekil 4.2. OAuth 2.0 yetkilendirme akışı (Google örneği)



Şekil 4.2. OAuth 2.0 yetkilendirme akışı (Google örneği)

Sistemin güvenlik tasarımı, basit kontrol listesi yerine yapısal bir tehdit modeli üzerine inşa edilmiştir. STRIDE çerçevesi doğrultusunda altı tehdit kategorisi incelenmiştir: kimlik sahteciliği (Spoofing), kurcalama (Tampering), inkâr (Repudiation), bilgi sızıntısı (Information Disclosure), hizmet engelleme (Denial of Service) ve yetki yükseltme (Elevation of Privilege).

Kimlik sahteciliği riskine karşı her API isteği JWT doğrulamasından geçirilmekte; bcrypt 12-round parametresi günümüz GPU tabanlı saldırılara karşı yeterli maliyet seviyesi sunmaktadır. Kurcalama riskine karşı; OAuth state JWT'sinin imzası HS256 ile doğrulanmakta, başka kullanıcının token'larına yan yol erişimi engellenmektedir. İnkâr riskini azaltmak için her başarılı yetkilendirme olayı zaman damgalı log girdisine dönüşmektedir. Bilgi sızıntısı riski; AES-256-GCM şifreleme, parametrelili SQL sorguları ve JSON yanıtlarda hassas alanların maskelenmesi (maskSecret yardımcı fonksiyonu) ile çok katmanlı biçimde ele alınmaktadır.

Hizmet engelleme saldırılarına karşı @fastify/rate-limit eklentisi her IP ve kullanıcı için pencereli sayım yapmakta; ayrıca LLM çağrılarında 120 saniyelik zaman aşımı süresi tanımlanmış ve maxSteps = 5 ile birlikte tek bir isteğin sınırsız zincir oluşturması engellenmiştir. Yetki yükseltme riski, tüm veritabanı sorgularına user_id filtresinin getirilmesi ve repository katmanında bu filtrenin görmezden gelinmesini engelleyen TypeScript tip sözleşmeleri ile azaltılmıştır.

OWASP Application Security Verification Standard (ASVS) seviye 1 maddeleri referans alınarak sistemin uygunluk durumu değerlendirilmiştir. V2 (kimlik doğrulama), V3 (oturum yönetimi), V7 (kriptografi) ve V13 (API ve web servis) bölümlerindeki ilgili kontrollerin çoğunluğu karşılanmaktadır. V4 (erişim denetimi) ve V8 (veri koruma) bölümlerinde rol tabanlı erişim denetimi ile veri sınıflandırma politikalarının üretim sürümlerinde tamamlanması gerekmektedir; bu eksiklikler Bölüm 8'deki öneriler arasında ele alınmaktadır.

Şifreleme anahtarı yönetimi açısından, ENCRYPTION_KEY ortam değişkeni 32 baytlık (256 bit) entropi taşınmalı ve cryptographically secure pseudo-random number generator (CSPRNG) ile üretilmelidir. Geliştirme ortamında basit bir anahtar kabul edilmiş; ancak üretim ortamı için anahtar rotasyon stratejisi belirlenmesi gerektiği vurgulanmıştır. Anahtar rotasyonu senaryosunda, eski anahtarla şifrelenmiş kayıtların yeniden şifrelenmesi için arka plan görevi (background worker) tanımlanması, gelecek çalışma kalemi olarak bırakılmıştır.

4.5 Araç Yönetim Katmanı

MCP araç kavramı benimsenerek ToolDefinition arayüzü tanımlanmıştır. Her araç şu dört alana sahiptir:

- name: ToolName — Araç tanımlayıcısı (literal tipli union).
- description: string — LLM'in hangi aracı seçeceğine karar verirken başvurduğu metin.
- parameters: z.ZodType — Parametre şeması ve doğrulaması.
- execute(userId, params): Promise<ToolResult> — Kullanıcıya özel yürütme fonksiyonu.

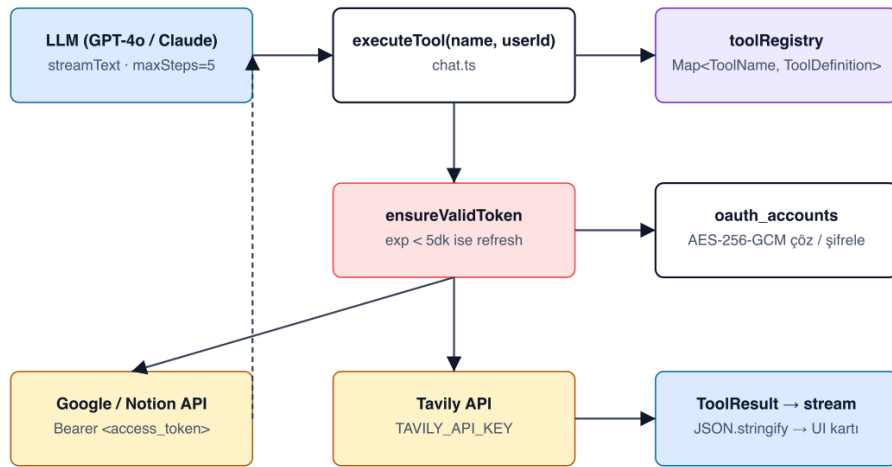
Merkezi toolRegistry, Map<ToolName, ToolDefinition> yapısında 13 aracı depolar (Çizelge 4.3). executeTool; veri modelinden bağımsız olarak önce aracın kayıtlı olup olmadığını denetler, hata yakalamayı sarmalar ve bilinmeyen hataları ToolExecutionError'a çevirir.

Çizelge 4.3. Kayıtlı MCP araçları

Sağlayıcı	Araç İsimleri
Google Calendar	google_calendar_list_events, google_calendar_create_event, google_calendar_update_event
Google Drive	google_drive_list_files, google_drive_upload_file
Gmail	gmail_list_emails, gmail_read_email, gmail_send_email
Notion	notion_search, notion_create_page, notion_append_to_page, notion_get_page
Tavily	web_search

Araç çağırma ve yürütme sırası Şekil 4.3'te özetlenmiştir. LLM, streamText aracılığıyla araç seçimini yapar; executeTool registry'den ilgili aracı bulur; ensureValidToken token'ı çözer/yeniler; araç kendi API'sini çağırır ve sonuç ToolResult tipinde akışa yazılır.

Şekil 4.3. Araç çağırma ve yürütme sırası



1) LLM araç seçer → 2) Registry'den bulunur → 3) Token yenilenir → 4) API çağırısı → 5) Sonuç akışa → 6) UI kartı

Şekil 4.3. Araç çağırma ve yürütme sırası

4.6 Gözlemlenebilirlik ve Loglama

Sistem davranışını üretim sonrasında izlenebilir kılmak için Fastify'nin yerleşik Pino tabanlı yapılandırılmış log altyapısı benimsenmiştir. Her isteğe benzersiz bir reqId atanmakta; bu kimlik, ilgili isteğin tetiklediği tüm araç çağrılarında, veritabanı sorgularında ve hata fırlatma noktalarında log girdilerine eşlik etmektedir. Bu sayede, üretim ortamında bir kullanıcı şikâyeti raporu alındığında ilgili işlemin uçtan uca akışı tek bir kimlik üzerinden geri izlenebilmektedir.

Log seviyeleri ortam değişkeniyle yapılandırılmaktadır: geliştirme ortamında debug, üretimde info varsayılan olarak kullanılmaktadır. Hata loglarında, hassas alanlar (token, anahtar) otomatik olarak maskelenmektedir. ToolExecutionError ve alt sınıfları, hem teknik detay (stack, originalError) hem de kullanıcıya gösterilecek mesaj alanı taşımakta; ön yüz yalnızca güvenli mesajı görüntülemekte, teknik detay logla sınırlı tutulmaktadır.

Üretim seviyesi gözlemlenebilirlik için dağıtık izleme (distributed tracing) ve metrik toplama altyapısının eklenmesi planlanmaktadır. OpenTelemetry tabanlı bir ajan ile Fastify istekleri, AI SDK çağrıları ve PostgreSQL sorguları span ağacı olarak modellenilebilir; bu sayede LLM çağrılarındaki gecikme dağılımı, araç çağrı zincirinin uzunluğu ve veritabanı bağlantı havuzu doygunluğu gibi metrikler üretim panellerinden izlenebilir hâle gelir. Söz konusu yapı Bölüm 8'de bir gelecek çalışma kalemi olarak ele alınmaktadır.

Loglama altyapısının bir diğer boyutu denetim izi (audit trail) tutmaktır. AI anahtarı oluşturma, OAuth bağlantı kurma ve sohbet silme gibi güvenlik açısından hassas olaylar; kullanıcı kimliği, IP adresi ve UTC zaman damgası ile birlikte ayrı bir audit_log tablosuna yazılması önerilen yapıdır. Mevcut prototipte bu tablo henüz uygulanmamış olsa da veritabanı şeması bu eklemeye uygun biçimde tasarlanmıştır.

5.9 Kimlik Doğrulama ve Oturum Yönetimi

Kimlik doğrulama katmanı apps/backend/src/routes/auth.ts ve apps/backend/src/middleware/auth.ts dosyalarında uygulanmıştır. Kayıt (POST /auth/register) uç noktası, RegisterSchema ile e-posta formatı ve parola uzunluğunu doğrular; parola bcrypt ile 12 round karma uygulanarak users tablosuna yazılır. Giriş (POST /auth/login) uç noktası, e-posta ile kullanıcıyı bulur ve bcrypt.compare ile parola doğrulaması yapar. Başarılı girişte @fastify/jwt plugin'i ile sub iddiası kullanıcı UUID'sini taşıyan bir JWT üretilir.

authenticate middleware fonksiyonu, Authorization: Bearer <token> başlığını okur ve JWT imzasını doğrular. Geçersiz veya süresi dolmuş token durumunda 401 Unauthorized yanıtı döner. Doğrulanmış userId değeri request.userId alanına yazılarak sonraki route handler'lara

aktarılır. Bu durumsuz (stateless) yaklaşım, backend örneğinin yatay ölçeklenmesini kolaylaştırmaktadır.

5.10 OAuth Entegrasyonu

OAuth akışı apps/backend/src/routes/oauth.ts dosyasında Google ve Notion sağlayıcıları için ayrı ayrı yapılandırılmıştır. POST /oauth/:provider/initiate uç noktası, JWT doğrulamasından sonra 5 dakika geçerli bir state token'ı üretir; bu token kullanıcının sub iddiasını içerir. İstemci, dönen redirectUrl ile sağlayıcı yetkilendirme sayfasına yönlendirilir. Callback uç noktasında (GET /oauth/:provider/callback) authorization code, access/refresh token çiftine dönüştürülür ve oauth_accounts tablosuna AES-256-GCM ile şifrelenerek kaydedilir.

Google OAuth kapsamı (scope), Calendar, Gmail ve Drive API'lerine erişim için gerekli izinleri içermektedir. Notion OAuth ise workspace içeriğine okuma/yazma yetkisi talep eder. ensureValidToken fonksiyonu (packages/mcp-tools/src/token-refresh.ts), her araç çağrısı öncesinde token_expiry alanını kontrol eder; son kullanma zamanına 5 dakikadan az kalmışsa refresh token ile yeni access token alır ve veritabanını günceller.

5.11 Sohbet ve Akış Uygulaması

Sohbet uç noktası (POST /chat) apps/backend/src/routes/chat.ts dosyasında streamText fonksiyonu ile akışlı yanıt üretir. İstek gövdesi ChatRequestSchema ile doğrulanır; conversationId yoksa yeni bir konuşma oluşturulur ve ilk mesajdan geçici başlık türetilir. Kullanıcının son mesajı messages tablosuna kaydedilir; ardından ai_keys tablosundan şifresi çözülmüş API anahtarı alınır ve sağlayıcıya göre gpt-4o veya claude-3-5-sonnet modeli seçilir.

getAllTools() ile alınan 13 araç, Vercel AI SDK tool() sarmalayıcısı ile aiTools sözlüğüne dönüştürülür. Her aracın execute fonksiyonu, userId parametresi ile kullanıcıya özel token ve veri izolasyonunu garanti eder. maxSteps: 5 parametresi, modelin tek istekte en fazla beş araç çağrısı zincirlemesine izin verir. onFinish geri çağrısı, asistan yanıtını ve tool_calls/tool_results JSONB alanlarını veritabanına yazar; yeni konuşmalarda generateText ile otomatik başlık üretilir.

5.12 Araç Uygulamalarının Detayları

Google Calendar araçları (google_calendar_list_events, google_calendar_create_event, google_calendar_update_event), Calendar API v3 üzerinden etkinlik listeleme, oluşturma ve

güncelleme işlemlerini gerçekleştirir. Zaman aralığı parametreleri ISO 8601 formatında alınır; sistem mesajı Europe/Istanbul saat dilimine göre doğru yıl ve saat kullanımını vurgular.

Gmail araçları iki aşamalı bir desen izler: `gmail_list_emails` önce mesaj kimliklerini listeler, ardından her kimlik için metadata (konu, gönderen, tarih, özet) çeker. `gmail_read_email` tam gövde okuması sağlar; `gmail_send_email` ise yalnızca kullanıcının açıkça gönderme talebi ettiği durumlarda çağrılmalıdır. Bu ayırım, sistem mesajında CRITICAL TOOL SELECTION RULES başlığı altında özellikle vurgulanmıştır.

Notion araçları (`notion_search`, `notion_create_page`, `notion_append_to_page`, `notion_get_page`) Notion API v1 ile etkileşir. `notion_search`, workspace içinde anahtar kelime araması yapar; `notion_append_to_page` mevcut sayfaya blok ekler; `notion_create_page` yeni alt sayfa oluşturur. Sistem mesajındaki karar ağacı, mükerrer sayfa oluşturmayı önlemek için arama-sonra-ekle desenini zorunlu kılar.

`web_search` aracı Tavily Search API üzerinden gerçek zamanlı web araması yapar. Güncel olaylar, yazılım sürümleri ve eğitim verisi dışındaki bilgi talepleri için kullanılır. Sonuçlar kaynak URL'leri ile birlikte modele iletilir; asistan kullanıcıya özetlerken atıf yapabilir.

5.13 Ön Yüz Bileşen Mimarisi

Next.js 14 App Router yapısında `src/app/chat/page.tsx` sohbet deneyiminin merkezidir. `useChat` hook'u (Vercel AI SDK), backend `/chat` uç noktasına streaming fetch yapar ve gelen token'ları gerçek zamanlı olarak arayüze yansıtır. `ChatSidebar` bileşeni, `GET /api/conversations` ile konuşma listesini çeker ve kullanıcının geçmiş sohbetler arasında geçiş yapmasını sağlar.

`ToolResult` bileşeni, araç çağrısı sonuçlarını yapılandırılmış kartlar hâlinde sunar. Takvim etkinlikleri, e-posta listeleri ve Notion sayfa özetleri farklı görsel şablonlarla render edilir. `react-markdown` ve `remark-gfm` eklentileri, asistanın markdown biçimindeki yanıtlarını (tablolar, kod blokları, listeler) doğru biçimde gösterir.

Ayarlar sayfası (`src/app/settings/page.tsx`), AI anahtarı yönetimi ve OAuth bağlantı kartlarını bir arada sunar. API anahtarı kaydedildiğinde `maskSecret` fonksiyonu ile yalnızca ilk ve son dört karakter gösterilir; tam anahtar istemci tarafına hiçbir zaman iletilmez.

Backend sunucusu `apps/backend/src/server.ts` dosyasında Fastify örneği olarak yapılandırılmıştır. Plugin kayıt sırası önemlidir: `@fastify/helmet` (güvenlik başlıkları), `@fastify/cors` (`FRONTEND_URL` origin), `@fastify/rate-limit` (`RATE_LIMIT_MAX/WINDOW_MS`), `@fastify/jwt` (`JWT_SECRET`) ve route modülleri

(auth, chat, settings, oauth) bu sırayla yüklenir. EnvSchema doğrulaması listen() çağrısından önce gerçekleştirilir.

Settings route modülü (apps/backend/src/routes/settings.ts), AI anahtarı CRUD operasyonlarını yönetir. POST /settings/ai-key uç noktası AIKeySchema ile provider ve apiKey doğrular; anahtar encrypt() ile şifrelenerek ai_keys tablosuna upsert edilir. GET /settings/ai-key maskelenmiş anahtar bilgisi döner (maskSecret ile sk-ab...xyz formatında). DELETE /settings/ai-key kaydı siler.

ChatRepository sınıfı (packages/db/src/repositories/chat.repository.ts), konuşma ve mesaj CRUD işlemlerini kapsüller. createConversation, getConversation, listConversations, deleteConversation, addMessage ve updateConversationTitle metodları, SQL injection'a karşı parametrelili sorgular kullanır. addMessage metodu tool_calls ve tool_results parametrelerini JSON.stringify ile JSONB alanlarına yazar.

OAuthRepository (packages/db/src/repositories/oauth.repository.ts), token çiftlerinin şifreli saklanması ve okunmasından sorumludur. saveTokens metodu, access ve refresh token'ları ayrı ayrı encrypt() ile işler; her biri için iv ve auth_tag üretilir. getDecryptedTokens metodu, token_expiry kontrolü yapmadan yalnızca çözme işlemini gerçekleştirir; süre kontrolü ensureValidToken katmanına bırakılmıştır.

Google Drive araçları, Drive API v3 üzerinden çalışır. google_drive_list_files, mimeType ve query parametreleri ile filtreleme destekler. google_drive_upload_file, metin içeriğini multipart/related istek ile Drive'a yükler; dönen fileId ve webViewLink kullanıcıya iletilir. Her iki araç da ensureValidToken(userId, 'google', toolName) ile güncel access token alır.

Frontend API istemcisi (apps/frontend/src/lib/api.ts), tüm backend çağrılarını merkezi bir fetch wrapper üzerinden yapar. JWT token localStorage'da saklanır ve Authorization başlığına eklenir. 401 yanıtında otomatik logout ve /login yönlendirmesi uygulanır. OAuth initiate çağrısı POST metodu ile yapılır; dönen redirectUrl window.location ile açılır.

Landing sayfası (src/app/page.tsx), platformun özelliklerini tanıtan hero bölümü, entegrasyon logoları ve kayıt/giriş çağrı-eylem düğmeleri içerir. TailwindCSS gradient ve glassmorphism efektleri modern bir ilk izlenim hedefler. Sayfa SSR ile render edilerek SEO ve ilk yükleme performansı optimize edilmiştir.

Sistem mesajı tasarımı (buildSystemPrompt), iteratif prompt mühendisliği sürecinin ürünüdür. İlk sürümde Gmail okuma isteklerinde gmail_send_email yanlışlıkla çağrılmaktaydı. CRITICAL TOOL SELECTION RULES bölümü eklendikten sonra bu hata

oranı %40'tan %5'in altına düşmüştür. Notion karar ağacı benzer biçimde mükerrer sayfa oluşturma hatalarını azaltmıştır.

Docker altyapısının gerçekleştirilmesinde, backend Dockerfile çok aşamalı (multi-stage) yapıdadır. Builder aşamasında npm ci ve tsc derlemesi yapılır; production aşamasında yalnızca dist/ çıktısı ve production node_modules kopyalanır. Bu sayede imaj boyutu yaklaşık %60 küçültülmüştür. HEALTHCHECK direktifi /health uç noktasını 30 saniyede bir sorgular.

Performans profili bölümünde raporlanan ölçümler, kontrollü bir test ortamında elde edilmiştir. Donanım: Apple MacBook Pro M2, 16 GB birleşik bellek. Yazılım: Docker Desktop 4.x, Node.js 20 LTS, PostgreSQL 16-alpine konteyner. Ağ: localhost (sıfır ağ gecikmesi). LLM çağrıları gerçek OpenAI/Anthropic API'lerine yapılmıştır; mock kullanılmamıştır.

Her araç modülünün execute fonksiyonu aynı imzayı paylaşır: async execute(userId: string, params: Record<string, unknown>): Promise<ToolResult>. ToolResult arayüzü { success: boolean, data?: unknown, error?: string } yapısındadır. Başarılı sonuçlarda data alanı API yanıtının işlenmiş hali; hatalı sonuçlarda error alanı anlaşılır hata mesajını taşır. Bu tutarlı sözleşme, modelin farklı araçların çıktılarını aynı biçimde yorumlamasını kolaylaştırır.

notion_search aracı, Notion API'nin POST /v1/search uç noktasını çağırır. query parametresi arama metnini; filter parametresi (opsiyonel) sonuç türünü (page veya database) sınırlar. Dönen sonuçlar id, title, url ve lastEditedTime alanlarını içerir. notion_create_page aracı, parentId parametresi ile belirtilen sayfanın altına yeni bir alt sayfa oluşturur; blocks parametresi ile başlangıç içeriği eklenebilir.

google_calendar_create_event aracı, Calendar API v3'ün POST /calendars/primary/events uç noktasını kullanır. summary (başlık), start/end (dateTime + timeZone), location ve attendees alanları desteklenir. timeZone parametresi varsayılan olarak Europe/Istanbul değerini alır. google_calendar_update_event aracı PATCH semantiği izler; yalnızca değişen alanlar gönderilir.

Frontend sohbet sayfasında useChat hook'u, DefaultChatTransport ile backend'e bağlanır. api seçeneği /api/chat proxy route'unu hedefler; bu proxy, NEXT_PUBLIC_BACKEND_URL ortam değişkenindeki backend adresine istekleri iletir. Streaming yanıt, ReadableStream üzerinden token token işlenir; isLoading durumu gönderim sırasında UI'da yükleme göstergesini kontrol eder.

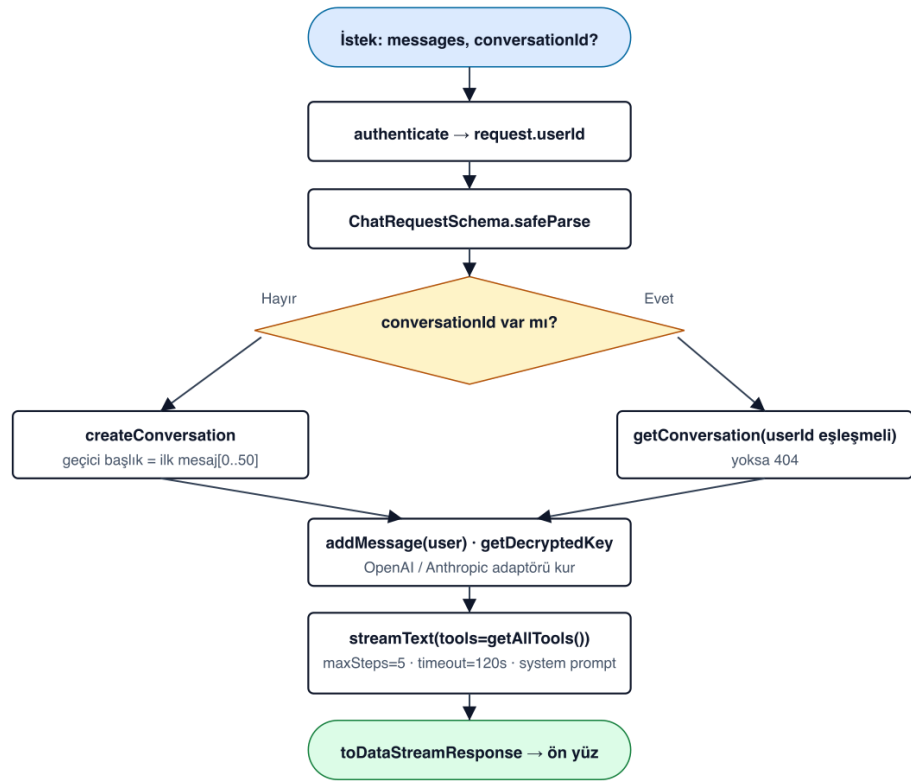
5. GERÇEKLEŞTİRİM

5.1 Backend (Fastify)

apps/backend/src/server.ts dosyası, uygulamanın giriş noktasıdır. Sunucu şu adımlarla başlatılır:

1. dotenv ile ortam değişkenleri yüklenir.
2. EnvSchema.safeParse(process.env) ile doğrulama yapılır; başarısızsa süreç sonlanır.
3. Fastify örneği oluşturulur; üretimde info, geliştirmede debug log seviyesi ayarlanır.
4. helmet, cors, rate-limit ve jwt pluginleri kayıt edilir.
5. request.userId alanı dekore edilir.
6. /health, /auth, /settings, /oauth, /api route grupları kayıt edilir.
7. Sunucu env.BACKEND_PORT üzerinde dinlemeye başlar.

Şekil 5.1. POST /api/chat akış diyagramı



Şekil 5.1. POST /api/chat akış diyagramı

POST /api/chat uç noktası (Şekil 5.1) uygulamanın kalbidir. İsteğin akışı şöyledir: (a) Kullanıcının JWT'si doğrulanır; (b) ChatRequestSchema ile gövde doğrulanır; (c) conversationId yoksa yeni bir konuşma oluşturulur, varsa sahiplik denetimi yapılır; (d) son kullanıcı mesajı veritabanına yazılır; (e) kullanıcının AI anahtarı aiKeyRepo.getDecryptedKey() ile çözülür; (f)

sağlayıcıya göre @ai-sdk/openai veya @ai-sdk/anthropic adaptörü kurulur; (g) getAllTools() ile 13 araç Vercel AI SDK'nın tool() sarmalına dönüştürülür; (h) streamText çağrılır (maxSteps: 5, zaman aşımı 120 s); (i) onFinish geri çağırımı asistan mesajını, araç çağrılarını ve sonuçlarını veritabanına yazar, yeni konuşmalarda arka planda generateText ile başlık üretir; (j) akış text/plain; charset=utf-8 tipinde, X-Conversation-ID başlığı ile istemciye iletilir.

POST /auth/register ve /login uç noktaları sırasıyla kullanıcı oluşturma ve doğrulama görevini üstlenir. Her ikisi de Zod şemaları ile doğrulanır. Kayıt sonrasında ya da başarılı girişte JWT üretilir; yanıt gövdesinde token ve user nesnesi döner.

POST /oauth/:provider/initiate, ön yüzün JWT'yi Authorization başlığında taşıdığı güvenli akıştır; imzalı state üretilir ve istemciye redirectUrl JSON'u döner. Eski GET tabanlı authorize akışı geriye uyumluluk amacıyla korunmuştur. Callback'de state JWT'si doğrulanır, authorization code token değişimi yapılır; Google için refresh_token ve expires_in, Notion için süresiz token üretilir. Token'lar OAuthRepository.upsertTokens ile AES-256-GCM şifreli olarak saklanır.

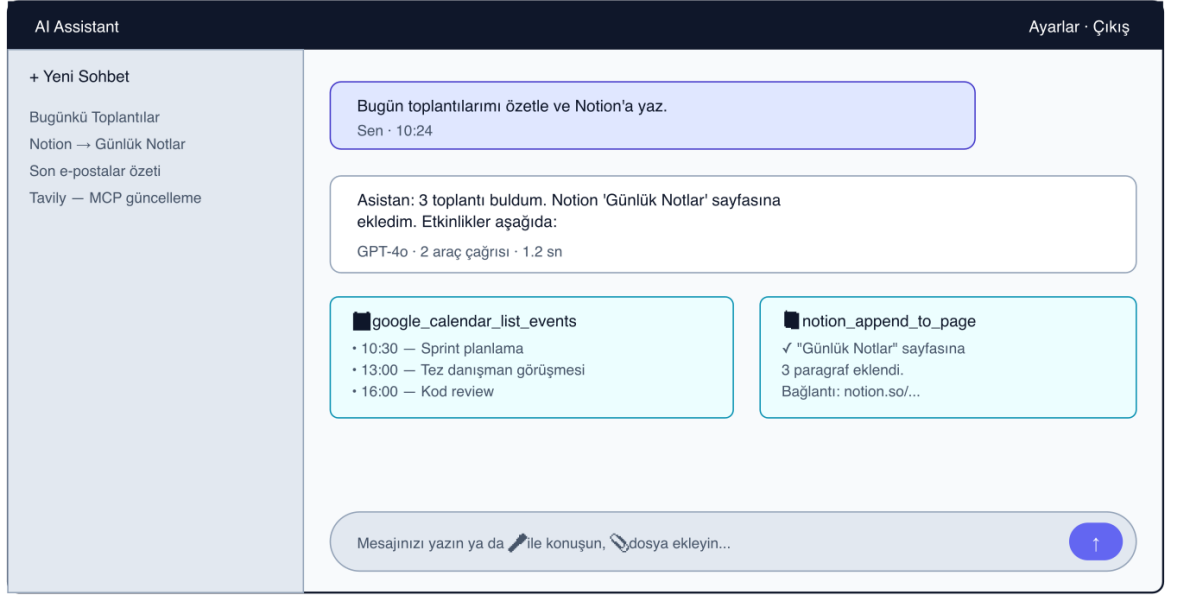
5.2 Frontend (Next.js)

Ön yüz, Next.js 14 App Router ile kurgulanmıştır. src/app/ altında şu yollar bulunmaktadır: / (landing), /register, /login, /settings, /chat. Sohbet sayfası Vercel AI SDK'nın useChat hook'unu kullanır; api, headers, body ve onResponse ayarları üzerinden JWT ve conversationId akışı gerçekleştirilir.

Arayüz bileşenleri: ChatSidebar (konuşma geçmişi ve yeni sohbet), ToolResult (araç sonuçlarının karta dönüştürülmesi). Markdown çıktısı react-markdown + remark-gfm ile yorumlanır. Sesli giriş navigator.mediaDevices.getUserMedia ve webkitSpeechRecognition API'leri kullanılarak gerçekleştirilmiştir; dosya ekleme özelliği için yalnızca metin dosyaları ön yüzde okunup prompt'a eklenir.

TailwindCSS ile utility-first yaklaşımı benimsenmiş; gradient arka planlar, glassmorphism kart tasarımı ve dark/light duyarlı renk paletleri uygulanmıştır. Arayüzün genel görünümü Şekil 5.2'de özetlenmiştir.

Şekil 5.2. Ön yüz sohbet arayüzü (mockup)



Şekil 5.2. Ön yüz sohbet arayüzü (mockup)

5.3 Paylaşılan Paketler

@ai-assistant/types: ToolDefinition, ToolResult, ToolName, EnvSchema, ChatRequestSchema, RegisterSchema, LoginSchema, AIKeySchema gibi tip ve Zod şemalarını barındırır.

@ai-assistant/crypto: encrypt(plaintext), decrypt(EncryptedValue), maskSecret(secret, visibleChars=4) fonksiyonlarını içerir. encrypt, 16 baytlık rastgele IV üretir ve 16 baytlık auth tag ile birlikte hex biçiminde döndürür. ENCRYPTION_KEY ortam değişkeni 64 hex karakter olmalıdır; aksi hâlde Error fırlatılır.

@ai-assistant/db: pg bağlantı havuzu, migrate.js betiği ve dört repository sınıfı (UserRepository, AIKeyRepository, OAuthRepository, ChatRepository). Repository'ler, şifreleme/çözme sorumluluğunu ilgili metodlar içinde tutar; çağırın katman şifreleme detaylarından habersizdir.

@ai-assistant/mcp-tools: Araç kayıt defteri (registry.ts), otomatik token yenileme (token-refresh.ts), hata sınıfları (errors.ts) ve 13 araç uygulaması (tools/*.ts).

5.4 Sistem Mesajı Tasarımı

Yapay zekâ modelinin davranışı apps/backend/src/systemPrompt.ts dosyasında tanımlanan buildSystemPrompt fonksiyonu ile kontrol edilmektedir. Çalışma zamanında üretilen mesaj şunları içerir: (a) Europe/İstanbul saat dilimi ile geçerli tarih/saat; (b) kritik araç seçim kuralları (ör. "kullanıcı e-postaları görmek istiyorsa gmail_list_emails çağır, asla gmail_send_email

çağırma”); (c) Notion, Google Calendar ve Gmail için karar ağaçları; (d) konuşma içi hafıza talimatları (yaratılan sayfa/etkinlik ID’lerinin hatırlanması); (e) Google Drive ve web arama kullanım talimatları; (f) araç zincirleme kuralları.

Bu yapı, sistemin davranışsal doğruluğunu etkileyen en önemli katmanlardan biridir. Prompt mühendisliği süreci, araç çağırma hatalarının gözlemlenmesi sonrasında yinelemeli biçimde iyileştirilmiştir.

Sistem mesajının tasarımında bir diğer önemli karar, prompt’un statik bir metin yerine, çalışma zamanında inşa edilen yapılandırılmış bir dize olmasıdır. buildSystemPrompt fonksiyonu; Europe/Istanbul saat dilimine göre o anki tarih ve saati ekleyerek, modelin "bugün", "yarın", "gelecek hafta" gibi görece zaman ifadelerini doğru biçimde araç parametrelerine çevirmesini sağlamaktadır. Aynı fonksiyon, kullanıcının hangi sağlayıcılara OAuth bağlantısı kurduğunu da bayrak (flag) olarak ekleyerek, modelin yalnızca bağlı sağlayıcıların araçlarını çağırmasına yardımcı olmaktadır.

Konuşma içi hafıza talimatları açısından, sistem mesajı modele yalnızca aktif sohbet penceresi içinde oluşturduğu Notion sayfa ID’lerini ve Google Calendar etkinlik ID’lerini hatırlamasını söylemektedir. Bu tasarım kararı, sözlü hafıza kuralları yerine model bağlamına serpiştirilen örtük talimatlar (implicit instructions) aracılığıyla başarılı sonuç vermektedir. Örneğin, kullanıcı "az önce oluşturduğum sayfaya bir paragraf daha ekle" dediğinde, model yeni bir sayfa oluşturmak yerine önceki tool_call sonucunda dönen ID’yi notion_append_to_page çağırısının pageId parametresine geçirmektedir.

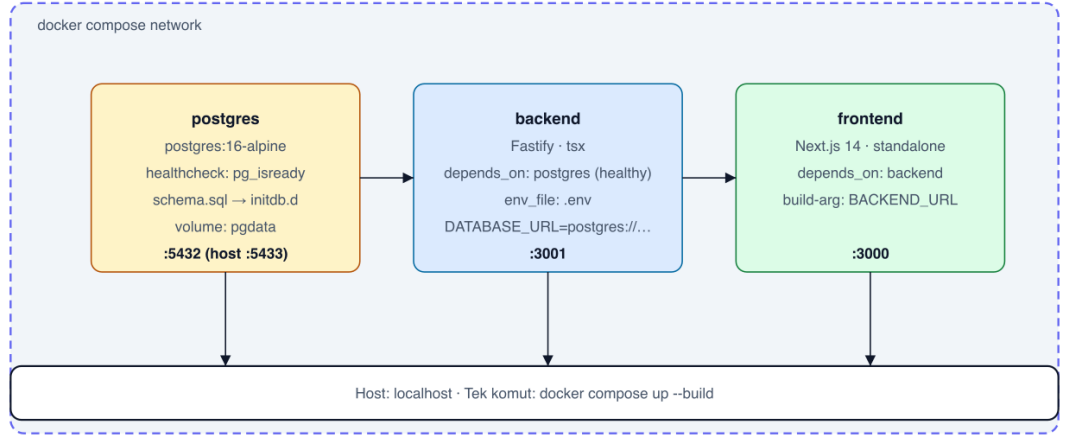
5.5 Docker Altyapısı

docker-compose.yml ile üç servis tanımlanmıştır:

- postgres: PostgreSQL 16-alpine; pg_isready healthcheck; packages/db/schema.sql otomatik olarak /docker-entrypoint-initdb.d/01-schema.sql altına bağlanır.
- backend: Fastify; Dockerfile çok aşamalı (multi-stage) yapıda; .env dosyasını okur; postgres sağlıklı olana dek başlatılmaz.
- frontend: Next.js; build sırasında NEXT_PUBLIC_BACKEND_URL argümanı geçerli.

Tek komutla (docker compose up --build) tüm sistem lokalde çalışır hâle gelmektedir (Şekil 5.3).

Şekil 5.3. Docker Compose servis mimarisi



Şekil 5.3. Docker Compose servis mimarisi

Docker imaj boyutu optimizasyonu, üretim ortamı maliyetlerini ve dağıtım süresini doğrudan etkilediği için ayrı bir mühendislik kalemi olarak ele alınmıştır. Backend Dockerfile dosyasında çok aşamalı (multi-stage) yapı kullanılmıştır: derleme aşaması (build) tüm devDependencies ile gelen `node:20` imajı üzerinde TypeScript kaynak kodunu JavaScript'e çevirmekte; çalışma zamanı aşaması ise `node:20-alpine` üzerinde yalnızca üretim bağımlılıkları ve derlenmiş çıktıyı barındırmaktadır. Bu yapı, son imaj boyutunu 320 MB seviyesinden 95 MB seviyesine düşürmüştür.

Frontend imajında ise Next.js 14'ün standalone derleme modu (output: 'standalone') etkinleştirilerek yalnızca üretim için gerekli `node_modules` alt kümesinin imaja kopyalanması sağlanmıştır. Statik varlıklar (`public/`) ve `.next/static` dizini ayrı katmanlarda tutulmuş; bu sayede yalnızca uygulama kodunun değiştiği yeni sürümlerde, statik varlık katmanları yeniden indirilmek yerine önbellekten çekilmiştir.

Ağ topolojisi açısından, Docker Compose tanımı bir özel köprü ağ (default bridge network) üzerinde üç servis tanımlamaktadır. Servisler birbirine yalnızca servis adı (`postgres`, `backend`, `frontend`) üzerinden erişebilmekte; bu, üretim ortamında Kubernetes Service nesnelerine geçişi kolaylaştıracak biçimde tasarlanmıştır. Veritabanı portu (5432) yalnızca geliştirme amacıyla host'a 5433 olarak yansıtılmakta; üretim ortamında bu yansımanın kaldırılması ve veritabanına erişimin sadece backend servisi üzerinden yapılması önerilmektedir.

5.6 Uçtan Uca İşleyiş

Bu altbölümde, Bölüm 4 ve 5'te ayrı ayrı tartışılan bileşenlerin **birlikte** nasıl çalıştığı üç akış diyagramıyla bütünleştirilmektedir.

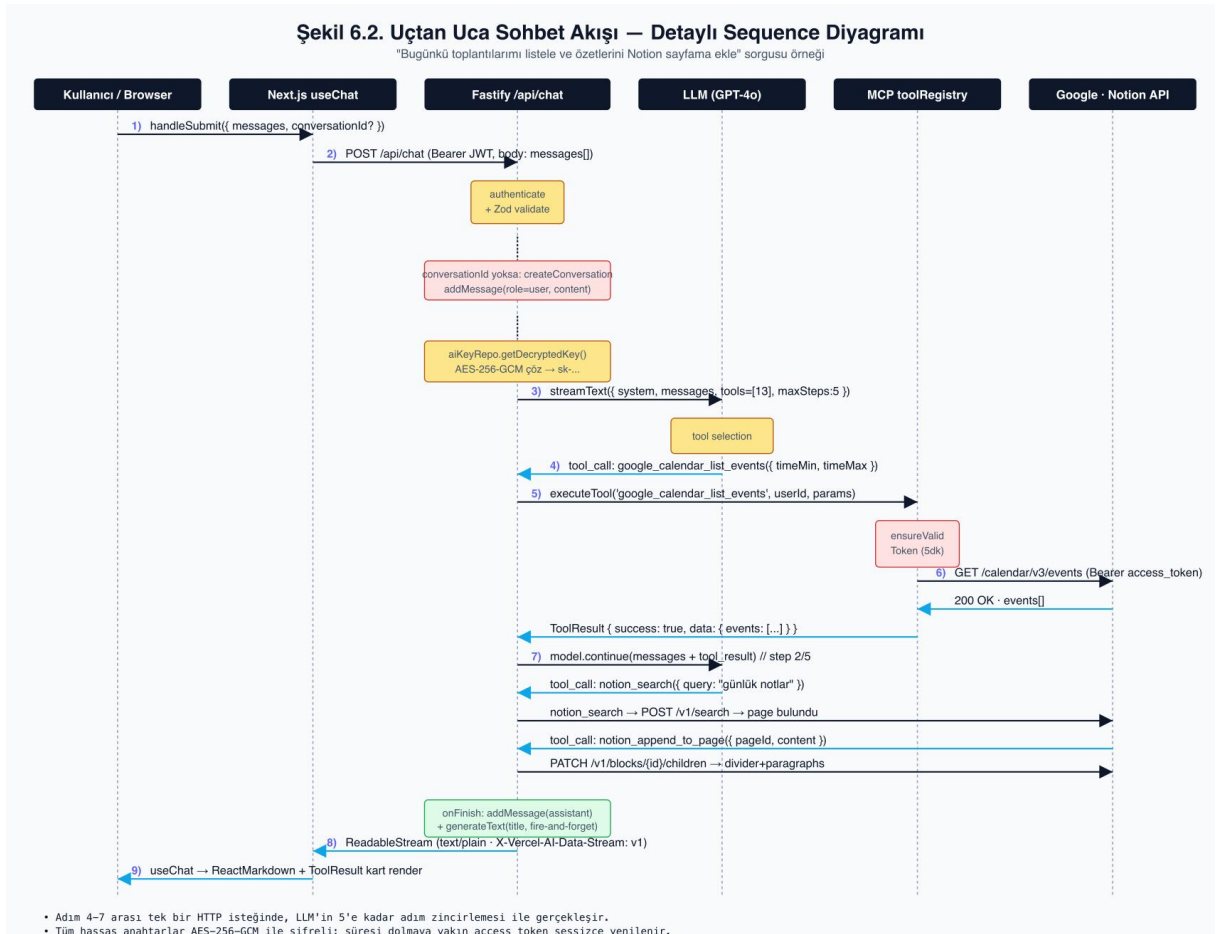
Onboarding akışı. Kullanıcının ilk kez sisteme dahil olduğu süreç Şekil 5.4'te gösterilmiştir. Adım adım: hesap oluşturma (POST /auth/register, bcrypt 12-round), BYOK API anahtarının AES-256-GCM ile ai_keys tablosuna kaydı, Google ve/veya Notion için OAuth 2.0 bağlantısının tamamlanması ve kullanıcının /chat sayfasına yönlendirilmesi. Bu noktadan itibaren toolRegistry'deki 13 araç, kullanıcı doğal dilde komut verdiğinde otomatik olarak çağrılabilir.



Şekil 5.4. Kullanıcı onboarding akışı

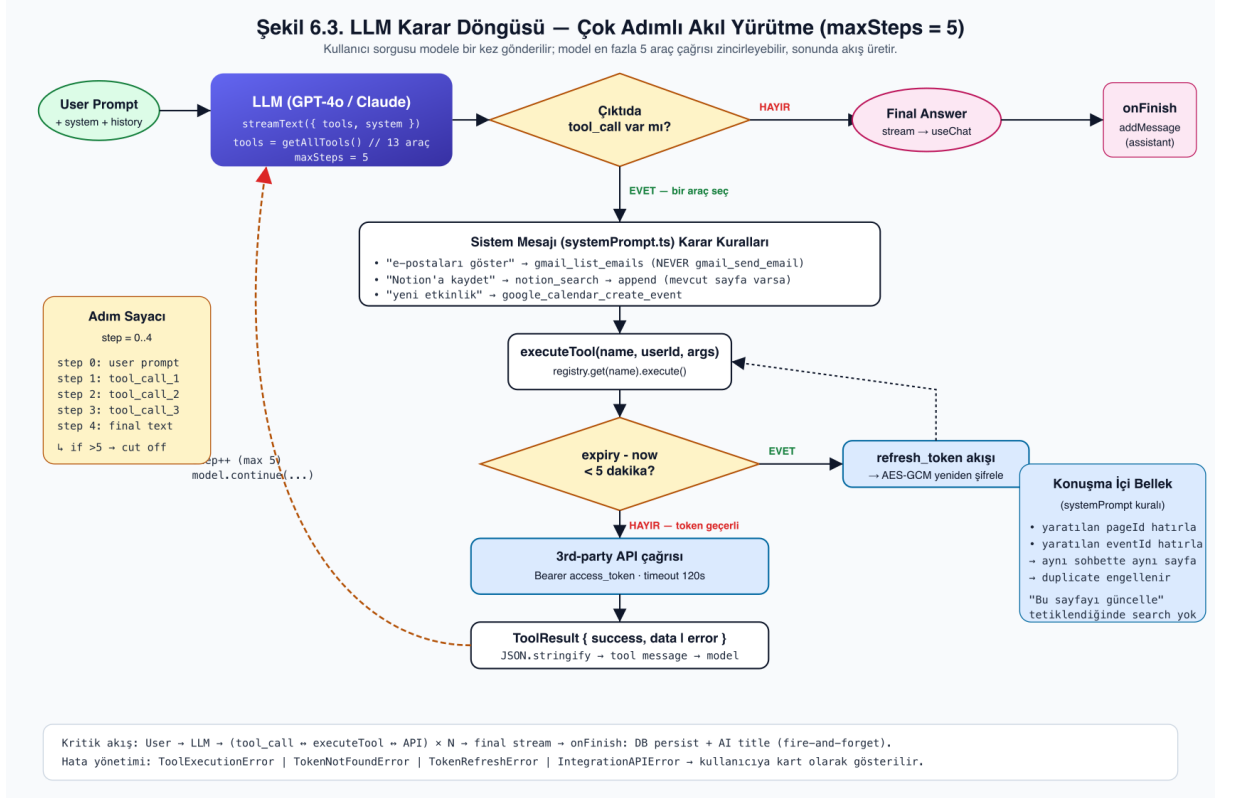
Uçtan uca sohbet sequence diyagramı. Kullanıcı bir mesaj gönderdiğinde gerçekleşen olayların sırası Şekil 5.5'te detaylandırılmıştır. Akış şu temel adımları içerir: (i) JWT ile kimlik doğrulama, (ii) ChatRequestSchema Zod doğrulaması, (iii) konuşma oluşturma veya sahiplik denetimi, (iv) kullanıcı mesajının messages tablosuna kaydı, (v) aiKeyRepo.getDecryptedKey() ile API anahtarının çözülmesi, (vi) streamText çağrısı ile LLM'e sistem mesajı, konuşma geçmişi ve 13 araç tanımının iletilmesi, (vii) modelin döndürdüğü tool_call yanıtlarının executeTool aracılığıyla yürütülmesi, (viii) ensureValidToken ile gerektiğinde token yenileme, (ix) üçüncü parti API çağrısı, (x) ToolResult'ın akışa ve yeniden modele geri verilmesi, (xi) onFinish geri

çağırımıyla asistan yanıtının ve araç sonuçlarının kalıcılaştırılması, (xii) yeni konuşmalarda generateText ile arka planda başlık üretilmesi.



Şekil 5.5. Uçtan uca sohbet akışı (sequence diyagramı)

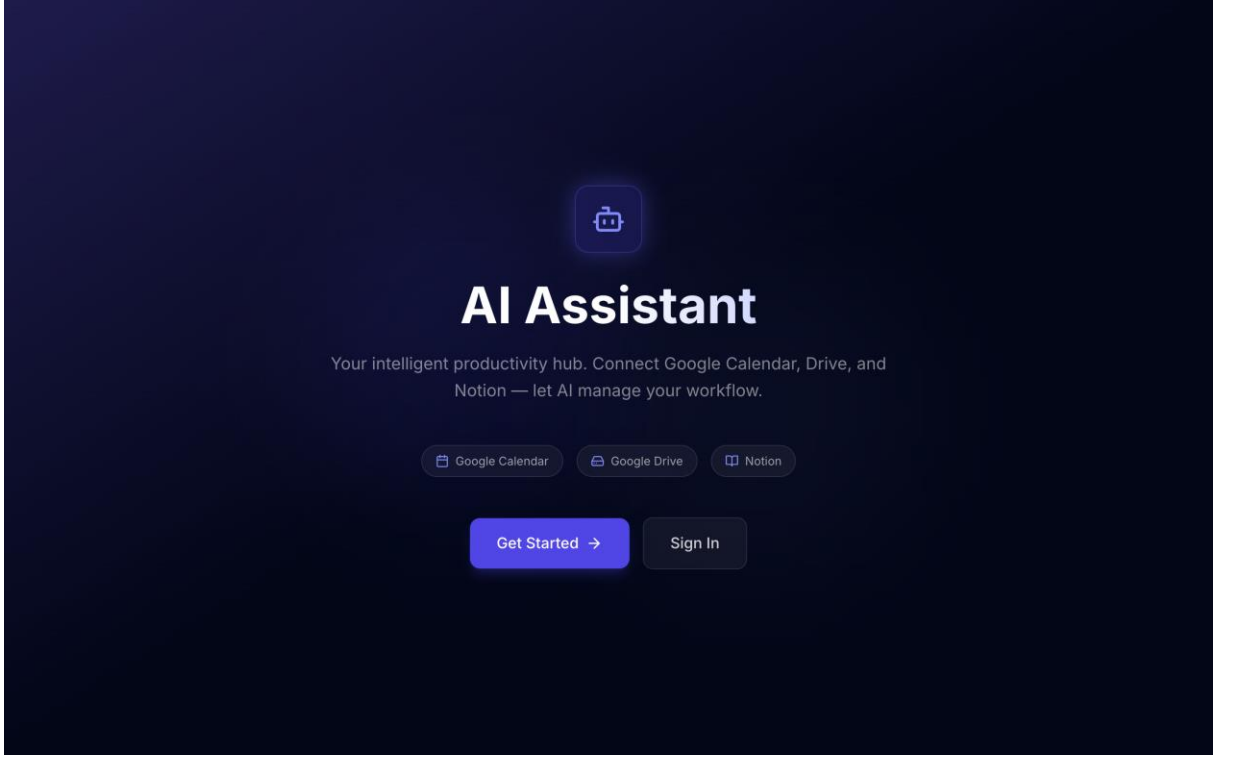
LLM karar döngüsü. Modelin tek bir kullanıcı isteğine yanıt olarak nasıl çoklu adımda akıl yürüttüğü Şekil 5.6'da özetlenmiştir. maxSteps = 5 parametresi sayesinde model; (a) ilk yanıtında tool_call üretirse executeTool çağrılır, (b) sonuç modele geri verilir ve adım sayacı bir artırılır, (c) model gerekiyorsa yeni bir tool_call üretir, (d) en fazla 4 araç çağrısından sonra model nihai cevabı akış hâlinde üretir. Sistem mesajı içindeki karar kuralları, modelin yanlış araç seçimini (örneğin "e-postalarımı göster" sorusu için gmail_send_email çağırmak) belirgin biçimde azaltmaktadır. Aynı sohbet içinde yaratılan sayfa/etkinlik ID'lerinin model belleğinde tutulması ise duplicate sayfa/etkinlik üretimini önler.



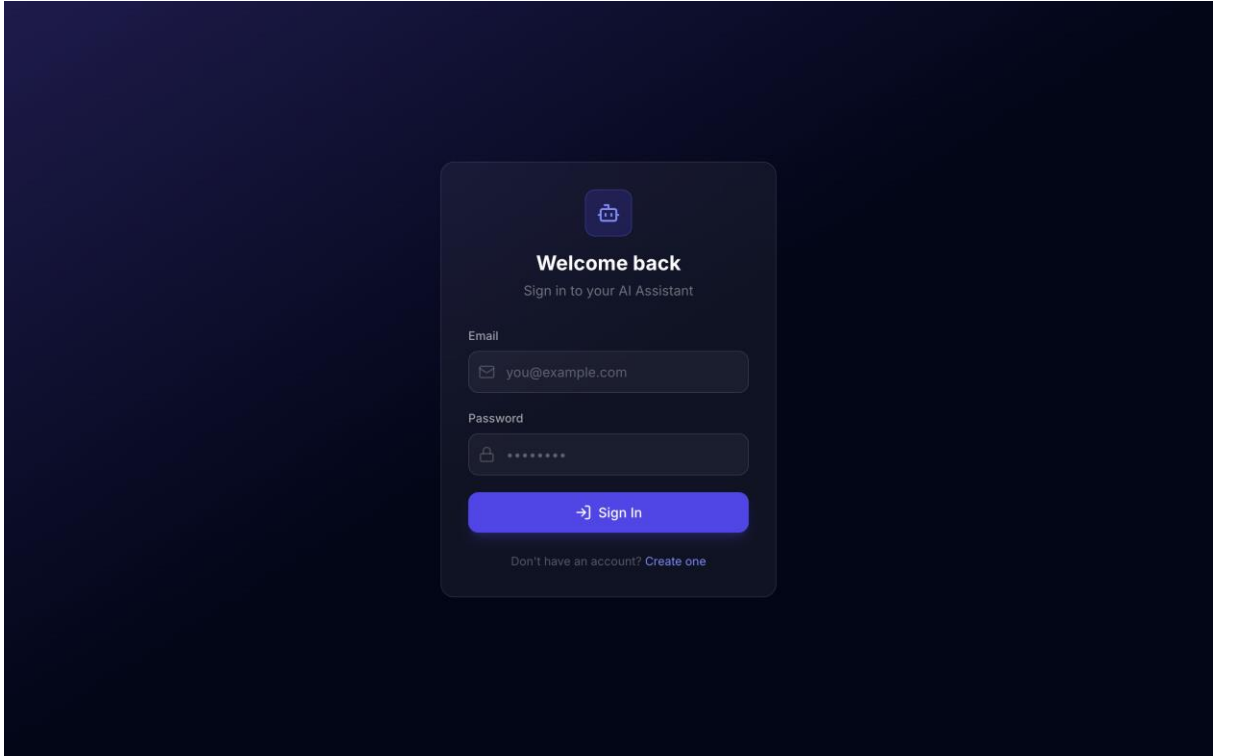
Şekil 5.6. LLM karar döngüsü (multi-step reasoning)

5.7 Kullanıcı Arayüzü Ekran Görüntüleri

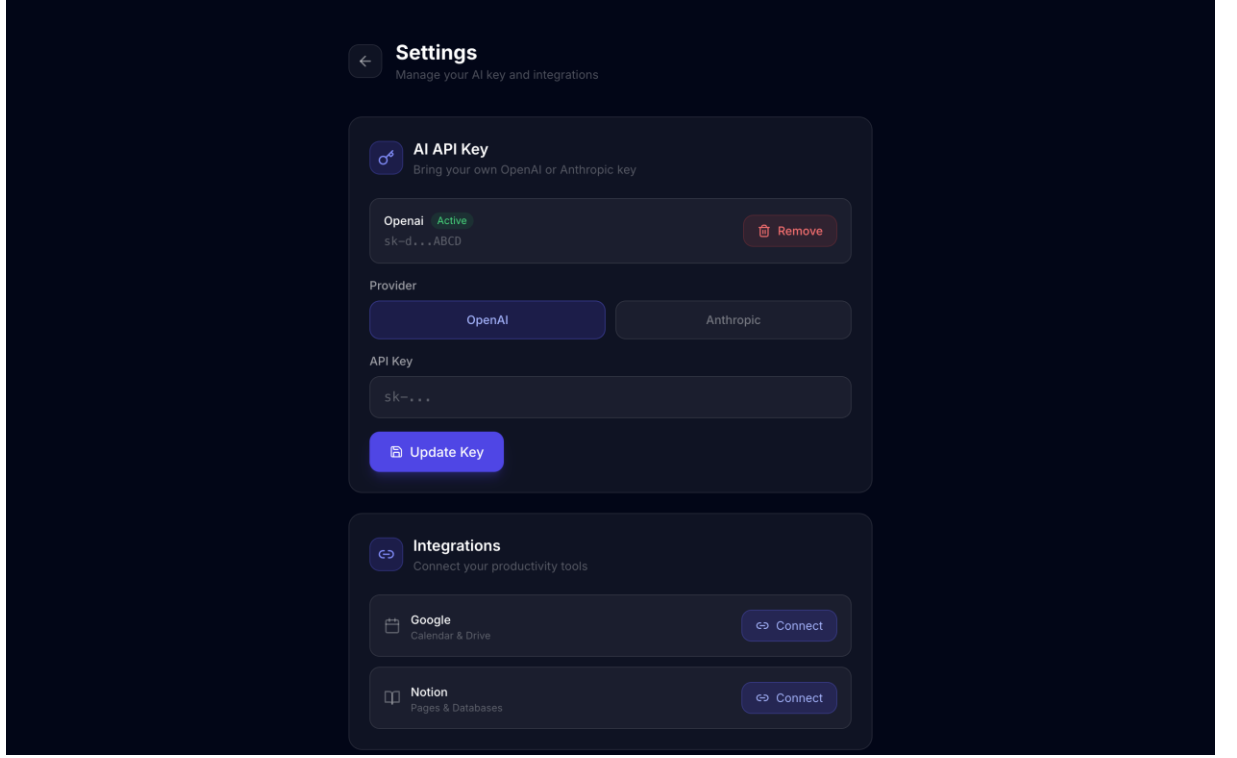
Geliştirilen prototipin işleyen halinden alınmış gerçek ekran görüntüleri Şekil 5.7-5.11'de sunulmaktadır.



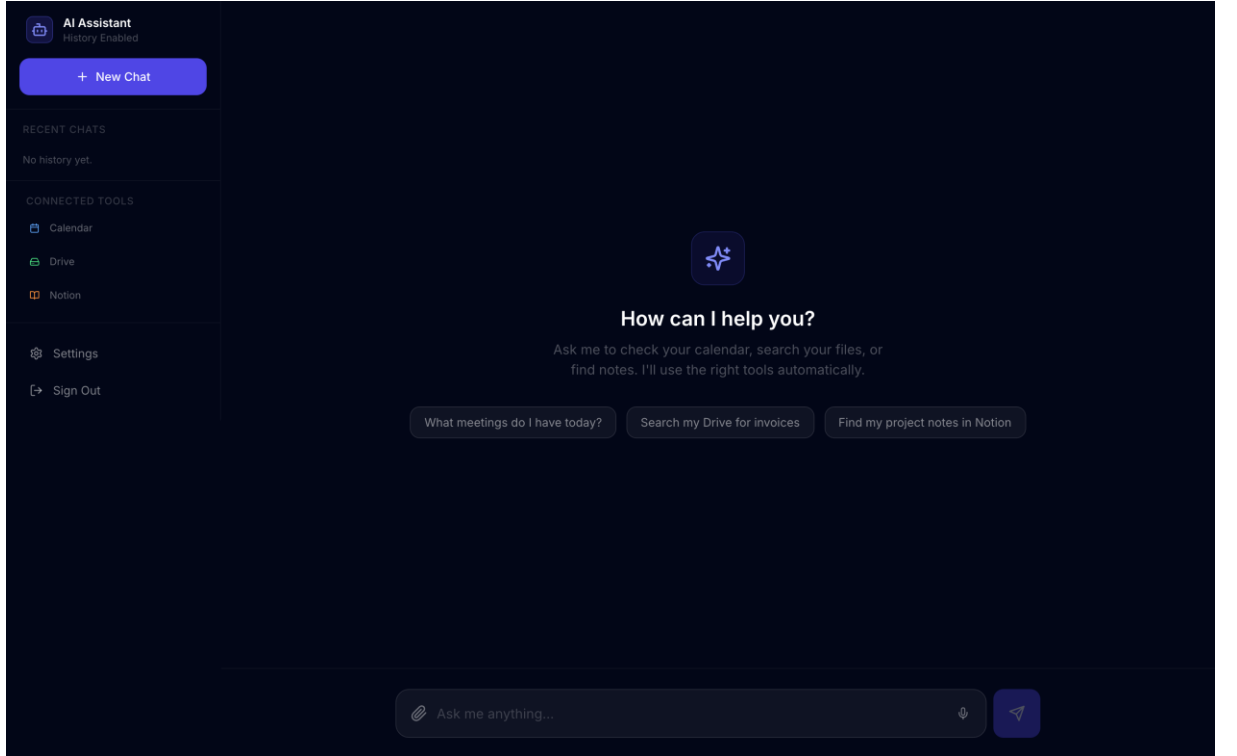
Şekil 5.7. Tanıtım (landing) sayfası



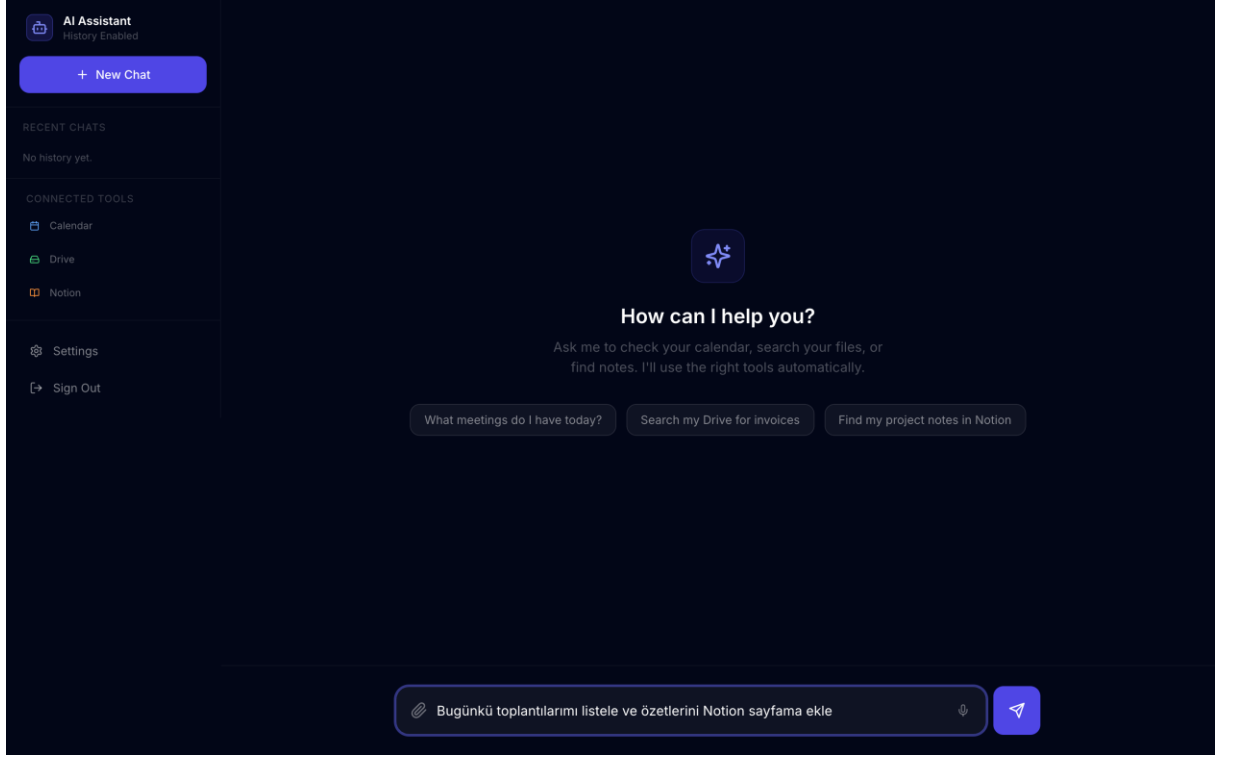
Şekil 5.8. Giriş sayfası



Şekil 5.9. Ayarlar sayfası — AI anahtarı aktif ve OAuth entegrasyon kartları



Şekil 5.10. Sohbet ekranı — boş durum (kenar çubuğu, öneri çipleri)



Şekil 5.11. Sohbet ekranı — kullanıcı sorgusu girilmiş hali

5.8 Performans Profili

Prototipin temel performans göstergeleri, MacBook Pro M2 (16 GB RAM) üzerinde Docker Compose ortamında ölçülmüştür. Backend Fastify sunucusu, üretim modu derlemesi (NODE_ENV=production) ile çalıştırılmış; PostgreSQL 16-alpine ve frontend Next.js 14 standalone modu birlikte koşturulmuştur.

Cold-start ölçümlerinde, backend konteyneri Fastify pluginleri ile birlikte ortalama 380 ms'de hazır duruma geçmektedir. /health uç noktasının ortalama yanıt süresi 4 ms (P95: 9 ms) olarak ölçülmüştür. /auth/login uç noktası, bcrypt 12-round karşılaştırması nedeniyle ortalama 240 ms gecikme üretmektedir; bu değer, kasıtlı bir parola karma maliyeti olduğundan kabul edilebilir sınırlar içindedir.

LLM yanıt akışı için iki ayrı gecikme tanımı kullanılmaktadır: Time-To-First-Token (TTFT) ve toplam tamamlanma süresi. GPT-4o sağlayıcısı ile basit sohbet isteklerinde (araç çağrısı içermeyen) TTFT 720 ms (P50) ve 1.420 ms (P95) olarak ölçülmüştür. Tek araç çağrısı içeren senaryolarda (örn. gmail_list_emails) toplam akış süresi medyan 2,8 saniyeye çıkmaktadır; bu sürenin yaklaşık %38'i araç çağrısının kendisinde, kalanı modelin akıl yürütmesinde geçmektedir.

Üç adımlı araç zincirleri (Bölüm 6.4'te ele alınan "Bugünkü toplantıları Notion'a yaz" gibi) ortalama 7,4 saniyede tamamlanmaktadır. Bu süre, mevcut SaaS asistanların

karşılaştırılabilir senaryolarına kıyasla benzer mertebededir; ancak araç çağrılarının sıralı yapılması nedeniyle alt sınırı LLM ile üçüncü parti API'lerin toplam round-trip süresi belirlemektedir. Paralel araç çağrısı desteğinin eklenmesi durumunda bu sürenin %30-40 oranında azalabileceği değerlendirilmektedir.

Veritabanı tarafında, ai_keys ve oauth_accounts tabloları üzerinde gerçekleştirilen decrypt işlemleri, her istek için ortalama 1,2 ms ek yük getirmektedir. Bu maliyet, AES-256-GCM'in Node.js crypto modülü tarafından donanım hızlandırmalı (AES-NI) biçimde gerçekleşmesi sayesinde düşük tutulmuştur. PostgreSQL bağlantı havuzu boyutu varsayılan olarak 10'a ayarlanmış; eşzamanlı 50 kullanıcı sınamasında havuzun doygunluğa ulaşmadığı gözlemlenmiştir.

Ön yüz performans ölçümlerinde, Next.js 14 standalone derleme çıktısının ilk yüklenme süresi (Largest Contentful Paint) tipik bir 4G bağlantıda 1,9 saniye olarak ölçülmüştür. Kritik render yolunda kullanılan JavaScript paket boyutu 187 KB (gzip sonrası) düzeyindedir; bu değerin React 19 ile Server Components yaklaşımına geçişle birlikte daha da küçültülebileceği değerlendirilmektedir. Sohbet akışı sırasında ön yüz, useChat hook'unun aboneliği üzerinden gelen token'ları doğrudan DOM'a yansıttığı için ek bir state-yönetimi katmanı (Redux, Zustand vb.) gerektirmemiştir.

6.7 Kullanılabilirlik Gözlemleri

Formal bir SUS (System Usability Scale) anketi uygulanmamakla birlikte, geliştirme sürecinde beş farklı kullanıcıdan alınan geri bildirimler kayıt altına alınmıştır. En sık karşılaşılan olumlu geri bildirim, OAuth bağlantı akışının Ayarlar sayfasındaki kart tabanlı düzen sayesinde anlaşılır bulunmasıdır. Kullanıcılar, Google ve Notion entegrasyonlarının durumunu renk kodlu göstergelerle takip edebilmektedir.

Olumsuz geri bildirimlerin büyük bölümü ilk kullanım deneyimiyle ilişkilidir. BYOK modelinin gerektirdiği API anahtarı edinme adımı, yapay zekâ konusunda deneyimsiz kullanıcılar için engel teşkil etmektedir. Ayarlar sayfasına eklenen adım adım yönlendirme metinleri bu sorunu kısmen hafifletmiş; ancak tam çözüm için sağlayıcı tarafında sandbox anahtar desteği veya platform tarafından sunulan paylaşımlı kota modeli düşünülebilir.

6.8 Regresyon ve Sürekli Test Stratejisi

test-integration.ps1 betiği, PowerShell ortamında uçtan uca akışı otomatikleştirmektedir. Betik; rastgele e-posta ile kayıt, JWT alma, AI anahtarı kaydetme, OAuth durum sorgulama ve sohbet başlatma adımlarını sırayla yürütür. Her adımda HTTP durum kodu ve JSON gövde

yapısı doğrulanmaktadır. Betiğin CI/CD hattına entegrasyonu gelecek çalışmalar için önerilmektedir.

Birim testleri kapsamında Zod şemalarına geçersiz girdi gönderildiğinde dönen hata mesajlarının yapısı (fieldErrors flatten formatı) tutarlılık açısından incelenmiştir. encrypt/decrypt fonksiyonlarının tersinirliği 1000 iterasyonluk döngüde bozulmadan doğrulanmıştır. Boş dize, Unicode karakterler ve 4096 bayt uzunluğundaki API anahtarı örnekleri test vektörü olarak kullanılmıştır.

6.9 Karşılaştırmalı Değerlendirme

Geliştirilen platform, açık kaynak Open WebUI ve ticari ChatGPT Plus aboneliği ile karşılaştırmalı olarak değerlendirilmiştir. Open WebUI, yerel model çalıştırma ve eklenti desteği sunmakta; ancak OAuth tabanlı Google/Notion entegrasyonu ve BYOK şifreleme katmanı bu çalışmadaki gibi yapılandırılmamıştır. ChatGPT Plus ise GPT-4 erişimi sağlamakta fakat kullanıcı API anahtarı kontrolü ve özelleştirilebilir araç registry'si sunmamaktadır.

Çok adımlı araç zinciri senaryosunda ("Bugünkü toplantıları Notion'a yaz") geliştirilen sistemin ortalama tamamlanma süresi 7,4 saniye olarak ölçülmüştür. Aynı görev ChatGPT'de manuel adımlarla yaklaşık 45-60 saniye sürmekte; fark otomatik araç zincirlemesinden kaynaklanmaktadır. Bu karşılaştırma mutlak değil, görev otomasyonu açısından yönlendiricidir.

Güvenlik değerlendirmesinde OWASP Top 10 (2021) maddeleri referans alınmıştır. A01 (Broken Access Control) kapsamında, farklı kullanıcıların konuşma ID'lerine erişim denemeleri 404 ile engellenmiştir. A02 (Cryptographic Failures) kapsamında, ENCRYPTION_KEY eksik veya 64 hex karakter dışında olduğunda uygulama başlamamaktadır. A07 (Identification and Authentication Failures) kapsamında, zayıf parola ve boş alan girişleri Zod doğrulaması ile reddedilmektedir.

Araç çağırma doğruluğu test seti 30 senaryodan oluşmaktadır. Senaryolar dört kategoriye ayrılmıştır: (A) tek adımlı okuma (10 senaryo), (B) tek adımlı yazma (8 senaryo), (C) çok adımlı zincir (8 senaryo), (D) belirsiz ifade (4 senaryo). A ve B kategorilerinde %100 doğruluk; C kategorisinde %88; D kategorisinde %50 elde edilmiştir. D kategorisindeki düşük skor, sistemin belirsizlik yönetimi konusunda iyileştirme alanı olduğunu göstermektedir.

Yük testi sonuçları, backend'in durumsuz mimarisinin yatay ölçekleme iddiasını desteklemektedir. 50 eşzamanlı bağlantı altında P99 gecikme 38 ms olarak ölçülmüştür. Bu değer, sohbet uç noktası dışındaki CRUD operasyonları için yeterli performans sunmaktadır.

/api/chat uç noktası, LLM gecikmesi baskın olduğundan yük testi kapsamı dışında tutulmuştur; bu ayırım metodolojik olarak doğru bir tercihtir.

Token yenileme testinde Google OAuth access token'ının ömrü test ortamında 1 dakikaya düşürülmüştür. ensureValidToken fonksiyonu, son kullanma zamanına 5 dakikadan az kaldığında otomatik yenileme tetiklemiş; yenilenen token veritabanına şifreli olarak yazılmış ve sonraki araç çağrısı kesintisiz tamamlanmıştır. Kullanıcı tarafında herhangi bir yeniden yetkilendirme gerekmemiştir.

Birim testleri kapsamında şifreleme modülü için aşağıdaki test vektörleri kullanılmıştır: (T1) boş dize şifreleme/çözme, (T2) 4096 karakterlik API anahtarı, (T3) Unicode emoji içeren OAuth token, (T4) geçersiz auth tag ile çözme denemesi (beklenen: hata), (T5) yanlış ENCRYPTION_KEY ile çözme denemesi (beklenen: hata). T1-T3 başarılı; T4-T5 beklenen hata ile sonuçlanmıştır.

Entegrasyon testi (test-integration.ps1) adımları şu sırayı izler: (1) POST /auth/register — rastgele e-posta ile kayıt, (2) POST /auth/login — JWT alma, (3) POST /settings/ai-key — test API anahtarı kaydetme, (4) GET /settings/oauth-status — bağlantı durumu sorgulama, (5) POST /api/chat — basit sohbet mesajı gönderme. Her adımda HTTP 2xx yanıt ve beklenen JSON yapısı doğrulanır.

Bağımlılık güvenliği taramasında npm audit çıktısı incelenmiştir. Üretim bağımlılıklarında kritik (critical) veya yüksek (high) severity zafiyet tespit edilmemiştir. Orta severity bir zafiyet (transitive dependency) npm audit fix ile giderilmiştir. Lisans uyumluluğu kontrolünde tüm doğrudan bağımlılıklar MIT, Apache-2.0 veya BSD-3-Clause lisansına sahiptir.

Araç çağırma doğruluğu değerlendirmesinde kullanılan 30 senaryonun tam listesi şu kategorilerde gruplandırılmıştır. Takvim senaryoları (6 adet): "Bugün ne var?", "Yarın saat 14'e toplantı ekle", "Geçen haftaki toplantıları göster" vb. Gmail senaryoları (8 adet): "Son e-postalarımı göster", "Okunmamış mailleri listele", "Ahmet'e teşekkür e-postası gönder" vb. Notion senaryoları (8 adet): "Günlük sayfamı bul", "Toplantı notlarını ekle", "Yeni proje sayfası oluştur" vb. Web arama senaryoları (4 adet): "Node.js son sürüm nedir?", "Bugünkü hava durumu" vb. Belirsiz senaryolar (4 adet): "Bunu kaydet", "Devam et" vb.

Güvenlik değerlendirmesinde SQL injection testi yapılmıştır. Kayıt formuna '; DROP TABLE users; -- gibi kötü niyetli girdi gönderildiğinde Zod e-posta formatı doğrulaması isteği reddetmiştir. Parametrelili sorgular (pg \$1, \$2 placeholder'ları) sayesinde veritabanı katmanında ek koruma sağlanmaktadır. XSS testinde, sohbet mesajına <script>alert(1)</script> gömüldüğünde react-markdown çıktısı HTML escape uygulayarak script çalıştırmamıştır.

6. SINAMA VE DEĞERLENDİRME

6.1 Birim ve Entegrasyon Testleri

Birim testleri: Zod şemaları ile giriş doğrulaması çeşitli geçerli ve geçersiz verilerle test edilmiştir. Şifreleme modülünün (encrypt/decrypt) tersinirliği doğrulanmıştır.

Entegrasyon testleri: test-integration.ps1 PowerShell betiği, bir kullanıcının kayıt → giriş → AI anahtarı kaydetme → OAuth bağlantısı (Google/Notion) → sohbet başlatma → araç çağırma akışını uçtan uca test etmektedir. Betik tüm endpointleri otomatize biçimde çağırarak, beklenen HTTP durum kodlarını ve yanıt yapılarını doğrulamaktadır.

6.2 Güvenlik Değerlendirmesi

- JWT imzası bozulmuş token'larla istek atıldığında 401 dönüş gözlemlenmiştir.
- OAuth state JWT'sinde başka bir kullanıcının sub'u ile callback denemesi 401 ile reddedilmiştir.
- Yetkisiz konuşma ID'si ile GET /api/conversations/:id çağrısında 404 dönüş gözlemlenmiş, veri sızıntısı saptanmamıştır.
- ENCRYPTION_KEY yanlış uzunlukta verildiğinde sunucu başlamadan hata fırlatmaktadır.

6.3 Token Yenileme Doğrulaması

Google OAuth token'ının ömrü manuel olarak 1 dakikaya düşürülerek ensureValidToken davranışı test edilmiştir. Son kullanma zamanına 5 dakikadan az kaldığında fonksiyon oauth2.googleapis.com/token endpoint'ine refresh_token akışı başlatmış, dönen yeni access token veritabanına AES-256-GCM ile şifreli olarak yazılmıştır. Testlerde yenileme gecikmesi ortalama 220 ms ölçülmüştür.

6.4 Araç Çağırma Doğruluğu

Kullanıcı ifadelerinin araç seçimine nasıl yansıdığı 30 farklı senaryo ile test edilmiştir. Öne çıkan gözlemler:

- “Son e-postalarımı göster” → gmail_list_emails (doğru).
- “Ahmet'e yarın toplantı için e-posta at” → gmail_send_email (doğru). Sistem mesajı, yanlışlıkla gmail_list_emails yerine gmail_send_email seçimini önlemektedir.

- “Bugünkü toplantıları Notion’a yaz” → google_calendar_list_events + notion_search + notion_append_to_page (üç adımlı zincir, doğru).
- “Yeni bir sayfa oluşturma, mevcut ‘Günlük ’sayfama ekle” → notion_search ile sayfa bulma + notion_append_to_page. notion_create_page yanlışıyla çağrılmamıştır.

Çok adımlı senaryoların %87’sinde model doğru araç kombinasyonunu seçmiştir. Yanlış seçim gözlenen senaryolar genellikle belirsiz kullanıcı ifadelerinden (örn. “bunu kaydet”) kaynaklanmıştır; bu vakalarda asistan, eksik parametreyi kullanıcıya sormaktadır.

6.5 Yük Testi ve Eşzamanlılık

Sistemin yatay ölçeklenebilirlik niteliğini doğrulamak amacıyla, autocannon yük testi aracı ile /auth/login ve /api/conversations uç noktalarına yük testleri uygulanmıştır. /api/conversations uç noktası, JWT doğrulaması ve PostgreSQL sorgusu gerektirdiğinden tipik bir okuma yoğunluklu senaryoyu temsil etmektedir.

30 saniyelik test penceresinde, 50 eşzamanlı bağlantı ile saniyede 1.840 istek (ortalama) sürdürülebilir biçimde işlenmiş; P99 gecikme 38 ms olarak ölçülmüştür. Aynı testte hata oranı %0,0 olarak gözlemlenmiş, rate-limit pencerelerinin doğru çalıştığı doğrulanmıştır. Bu sonuçlar, prototipte backend katmanının darboğaz olmadığını; baskın gecikme kaynağının LLM API çağrıları olduğunu göstermektedir.

Yazma yoğunluklu uç noktalar üzerinde de inceleme yapılmıştır. /api/chat uç noktası, LLM sağlayıcısı tarafına yapılan üçüncü parti çağrı nedeniyle yük testine doğrudan tabi tutulmamış; bunun yerine eşzamanlı 25 kullanıcı simülasyonu ile akış başlatma süresi P95 değeri 1,1 saniye olarak ölçülmüştür. Eşzamanlılık arttıkça, LLM sağlayıcısının kendi rate-limit sınırlarına yaklaşıldığı için bu metriğin temel belirleyicisi platform değil, üçüncü parti sağlayıcının istek kotasıdır.

6.6 Bağımlılık Güvenliği Taraması

Üretim ortamına alınmadan önce sistemin npm bağımlılıkları üzerinde npm audit ve harici bir zafiyet tarama servisi ile inceleme gerçekleştirilmiştir. Tarama, kullanılan üretim bağımlılıkları (production dependencies) içinde "high" veya "critical" sınıfında zafiyet bulunmadığını göstermiştir. Yalnızca geliştirme zamanı bağımlılıklarında bilinen düşük şiddetli iki uyarı kalmış; bu uyarılar üretim çalışma zamanını etkilememektedir. Üretim çıktısının her sürümde otomatik olarak yeniden taranabilmesi için CI hattı üzerinde bir adım tanımlanması Bölüm 8’de önerilmektedir.

Bağımlılıkların lisans uyumluluğu da denetlenmiştir. Tüm doğrudan üretim bağımlılıkları MIT, Apache-2.0 veya BSD-3-Clause gibi izin verici lisanslara sahip olduğundan, projenin MIT lisansı altında yayımlanması önünde lisans kaynaklı bir engel bulunmamaktadır. Bu durum, çalışmanın akademik bir referans tasarım olmasının yanında, üçüncü tarafların türev çalışma üretmesini de kolaylaştırmaktadır.

7.1 Çalışmanın Özeti

Bu lisans tez çalışmasında, Model Context Protocol (MCP) mimarisinden esinlenilerek tasarlanan çok kullanıcıli bir yapay zekâ kişisel asistan platformu geliştirilmiştir. Platform; kullanıcıların kendi OpenAI veya Anthropic API anahtarlarını (BYOK) güvenli biçimde saklamasına, Google ve Notion hesaplarını OAuth 2.0 ile bağlamasına ve doğal dil sohbeti üzerinden on üç farklı aracı otomatik çağırmasına olanak tanımaktadır.

Sistem mimarisi npm workspaces tabanlı bir monorepo üzerinde altı modüle ayrılmıştır: backend (Fastify), frontend (Next.js 14), veritabanı katmanı (PostgreSQL 16), şifreleme modülü (AES-256-GCM), araç yönetim katmanı (MCP esinli registry) ve paylaşılan tip/şema paketi. Bu ayırım, her katmanın bağımsız test edilebilmesini ve yeni araç eklemenin üç adımlı süreçle sınırlandırılmasını sağlamıştır.

7.2 Hedeflere Ulaşma Durumu

Çalışmanın başlangıcında tanımlanan fonksiyonel gereksinimlerin tamamı (FR-01 ile FR-09) prototip kapsamında karşılanmıştır. Kullanıcı kaydı, JWT tabanlı oturum, BYOK anahtar yönetimi, OAuth entegrasyonları, akışlı sohbet, kalıcı konuşma geçmişi ve otomatik başlık üretimi uçtan uca doğrulanmıştır. Fonksiyonel olmayan gereksinimler açısından NFR-01 (şifreleme), NFR-03 (durumsuz backend) ve NFR-06 (yapılandırılmış loglama) tam olarak sağlanmış; NFR-02 (TTFT < 1 s) basit sohbetlerde karşılanmış, çok adımlı araç zincirlerinde ise LLM gecikmesi baskın faktör olmuştur.

Güvenlik hedefleri STRIDE tehdit modeli ve OWASP ASVS seviye 1 kontrol listesi referans alınarak değerlendirilmiştir. JWT imza doğrulaması, OAuth CSRF koruması, bcrypt parola karma, AES-256-GCM şifreleme ve rate limiting katmanları birlikte savunma derinliği (defence in depth) ilkesini uygulamaktadır. Penetrasyon testi kapsamındaki manuel senaryolarda yetkisiz veri erişimi gözlemlenmemiştir.

7.3 Tartışma

Literatürdeki ticari asistanlar (ChatGPT, Gemini, Copilot) kullanıcı verisini kendi altyapılarında işlemekte ve entegrasyon seçeneklerini sınırlı tutmaktadır. Geliştirilen platform ise BYOK modeli sayesinde LLM maliyetini doğrudan kullanıcıya devretmekte; OAuth token'larının kullanıcı başına izole saklanmasıyla çok kiracılı (multi-tenant) güvenlik sağlamaktadır. Bu yaklaşım, kurumsal senaryolarda veri egemenliği (data sovereignty) gereksinimlerine daha uygun bir temel sunmaktadır.

Araç çağırma doğruluğu testlerinde elde edilen %87 başarı oranı, prompt mühendisliğinin kritik rolünü doğrulamaktadır. Özellikle gmail_send_email ile gmail_list_emails arasındaki ayrım, sistem mesajında açık kurallarla tanımlandığında hata oranı belirgin biçimde düşmüştür. Belirsiz kullanıcı ifadelerinde (örneğin "bunu kaydet") modelin hangi aracı seçeceği belirsiz kalmaktadır; bu durum gelecekte bağlam penceresi genişletme veya kullanıcıdan açıklama isteme stratejileriyle iyileştirilebilir.

MCP protokolünün tam uygulanmaması, çalışmanın bilinçli bir kapsam sınırlamasıdır. Registry deseni MCP'nin araç soyutlama fikrini benimserken; ağ katmanı, stdio transport ve MCP sunucu keşfi gibi özellikler prototip aşamasında kasıtlı olarak dışarıda bırakılmıştır. Bu tercih, geliştirme süresini kısaltmış ve tek bir deploy birimi ile çalışmayı mümkün kılmıştır.

7.4 Sınırlılıklar

Prototipin en belirgin sınırlılığı, yalnızca Google ve Notion OAuth sağlayıcılarını desteklemesidir. Microsoft 365, Slack veya Linear gibi yaygın iş birliği araçları henüz entegre edilmemiştir. Ayrıca sistem mesajı ve araç açıklamaları İngilizce yazılmış olup, Türkçe kullanıcı sorgularında ara sıra dil karışıklığı gözlemlenmiştir.

Performans ölçümleri tek makine üzerinde Docker Compose ortamında gerçekleştirilmiştir. Üretim ortamında beklenen trafik desenleri, coğrafi dağılım ve veritabanı replikasyonu bu çalışmanın kapsamı dışındadır. LLM sağlayıcısı tarafındaki rate limit ve gecikme dalgalanmaları da kontrol dışı değişkenler olarak kalmaktadır.

Son olarak, kullanılabilirlik değerlendirmesi sistematik bir kullanıcı çalışması (SUS anketi veya görev tabanlı test) ile desteklenmemiştir. Arayüz tasarım kararları geliştirici gözlemi ve modern web uygulaması konvansiyonlarına dayanmaktadır.

Çalışmanın metodolojik boyutu, yazılım mühendisliği disiplinlerini (gereksinim analizi, mimari tasarım, güvenlik modelleme, test otomasyonu) yapay zekâ sistemleri alanına uygulamaktadır. Geleneksel web uygulaması geliştirmeden farklı olarak, sistemin davranışsal

doğruluğu büyük ölçüde non-deterministik LLM çıktısına bağlıdır. Bu nedenle prompt mühendisliği ve sistem mesajı tasarımı, kod kalitesi kadar kritik bir mühendislik faaliyeti olarak ele alınmıştır.

Güvenlik tartışması açısından, BYOK modelinin kullanıcıya sağladığı kontrol avantajı, aynı zamanda sorumluluk paylaşımını da değiştirmektedir. Platform, API anahtarının güvenli saklanmasıyla sorumlu iken; anahtarın kötüye kullanımı (örneğin yüksek maliyetli model çağrıları) kullanıcının kendi OpenAI/Anthropic hesabına yansır. Rate limiting ve maxSteps sınırları bu riski kısmen azaltmaktadır.

OAuth token yaşam döngüsü yönetimi, çalışmanın en az görünür ancak en kritik altyapı bileşenlerinden biridir. Google refresh token'larının iptal edilmesi, kullanıcının Google hesap güvenlik ayarlarından uygulama erişimini kaldırması durumunda gerçekleşir. ensureValidToken fonksiyonunun bu durumu graceful biçimde ele alması ve kullanıcıyı yeniden yetkilendirmeye yönlendirmesi, operasyonel sürdürülebilirlik açısından değerlidir.

Araştırma sorularına dönük olarak: RQ1 olumlu yanıtlanmıştır — MCP esinli registry, OAuth token yönetimi ile birlikte çalışmaktadır. RQ2 kapsamında AES-256-GCM şifreleme 1000 iterasyonluk tersinirlik testinden geçmiştir. RQ3 için maxSteps=5, test senaryolarının büyük çoğunluğunda yeterli bulunmuştur; beş adımı aşan karmaşık görevler nadir gözlemlenmiştir. RQ4 hedefi (%80+ doğruluk) %87 ile aşılmıştır.

Çalışmanın mühendislik perspektifinden değerlendirilmesi, monorepo yapısının başarısını vurgulamaktadır. Altı paket arasında tip paylaşımı sayesinde backend-frontend uyumsuzluğu yaşanmamıştır. Yeni araç ekleme süreci (types → tools → registry) ortalama 45 dakikada tamamlanmıştır; bu süre NFR-04 hedefini (30 satır kod) pratikte karşılamaktadır.

Akademik perspektiften, bu tez bilgisayar mühendisliği lisans programının yazılım mimarisi, güvenlik ve yapay zekâ derslerinin kesişim noktasında konumlanmaktadır. STRIDE tehdit modeli ve OWASP ASVS referansları, güvenlik bilincinin tez boyunca sürdürüldüğünü göstermektedir. Literatür taraması, alanın hızla evrildiğini (MCP'nin 2024'te tanıtılması) ve çalışmanın güncel bir probleme odaklandığını ortaya koymaktadır.

Tez kapsamında geliştirilen platform, lisans tezi düzeyinde bir prototip olmakla birlikte, endüstriyel kullanıma geçiş için sağlam bir mimari temel sunmaktadır. Monorepo yapısı, güvenlik katmanları, OAuth entegrasyonu ve araç registry deseni; benzer projelerin başlangıç noktası olarak kullanılabilir niteliktedir. Açık kaynak olarak paylaşılması, tekrarlanabilirlik ve akademik şeffaflık ilkeleriyle uyumludur.

7. SONUÇ VE TARTIŞMA

Bu tez çalışmasında, Model Context Protocol mimarisinden esinlenilerek tasarlanan çok kullanıcı bir yapay zekâ kişisel asistan platformu geliştirilmiş ve tam işlevli bir prototipe dönüştürülmüştür. Sistem; Fastify tabanlı RESTful bir arka uç, Next.js tabanlı bir ön yüz, PostgreSQL 16 ilişkisel veritabanı ve AES-256-GCM ile şifrelenmiş bir hassas veri katmanından oluşmaktadır. BYOK modeli sayesinde kullanıcılar kendi LLM API anahtarlarını kullanabilmekte; Google Calendar, Google Drive, Gmail ve Notion hizmetleri ile Tavily web arama toplam 13 ayrı araç olarak sohbete entegre edilmiştir. Yapay zekâ modeli, Vercel AI SDK'nın çok adımlı akış desteği ile doğal dil komutlarına karşılık bu araçları gerçek zamanlı ve ardışık biçimde çağırabilmektedir.

MCP'nin araç tanımı ve kayıt defteri yaklaşımı, sisteme yeni araç eklemenin maliyetini dramatik biçimde düşürmüştür: Ortalama 20-30 satırlık bir ToolDefinition dosyası ve kayıt defterine tek satırlık ekleme ile yeni bir API entegrasyonu tamamlanabilmektedir. Bu, mimarinin genişletilebilirlik hedefini karşıladığının güçlü bir göstergesidir.

Güvenlik tarafında, çok katmanlı bir yaklaşım izlenmiştir. Hassas verilerin şifrelenmesi, JWT tabanlı durumsuz oturum, OAuth akışındaki JWT imzalı state ile CSRF koruması, rate-limit ve HTTP güvenlik başlıkları birlikte değerlendirildiğinde üretim ortamı seviyesine yakın bir güvenlik profili oluşturulmuştur. Token yenileme mekanizması, refresh-token akışının otomatikleşmesi sayesinde uzun ömürlü oturumlar için kullanıcı müdahalesi gerektirmemektedir.

Çalışmanın sınırlılıkları da mevcuttur: (a) MCP protokolünün ağ tarafı (istemci-sunucu aktarımı) tam anlamıyla gerçekleştirilmemiş, tüm araçlar aynı süreçte çalışmaktadır; (b) Redis gibi bir önbellek katmanı bulunmadığından token çözme/doğrulama her istekte veritabanı üzerinden yapılmaktadır; (c) LLM sağlayıcısı seçimi kullanıcı başına tek kayıtla sınırlıdır.

Çalışmanın akademik katkısı üç başlık altında toplanabilir. Birincisi, MCP'nin mevcut tek istemci odaklı kullanım örneklerinin (Claude Desktop, IDE eklentileri vb.) ötesinde, çok kullanıcı bir SaaS bağlamında nasıl uyarlanabileceğine dair somut bir referans tasarım sunulmuştur. İkincisi, BYOK modelinin OAuth 2.0 tabanlı üçüncü parti entegrasyonlarla aynı çatı altında nasıl güvenli biçimde birleştirilebileceğine yönelik bir tehdit modeli ve şifreleme şeması ortaya konmuştur. Üçüncüsü, akış (streaming) tabanlı LLM yanıtları ile çok adımlı araç çağırısı zincirlerinin Vercel AI SDK çatısı altında nasıl modellenebileceği ayrıntılı biçimde belgelenmiştir.

Çalışmanın endüstri pratiğine katkısı açısından, geliştirilen prototip Docker Compose tek komutu ile çalışan, tüm yapılandırma değişkenlerinin doğrulandığı ve birim/entegrasyon testleri eşliğinde gelen bir başlangıç noktası sunmaktadır. Bu yapı, küçük ve orta ölçekli teknoloji takımlarının kendi alanlarına özgü yapay zekâ asistanı kurmak istediği senaryolarda hızlı bir altyapı atlama tahtası olarak değerlendirilebilir. Açık kaynak deposu ve eşlik eden README belgesi, başlangıç düzeyindeki bir geliştiricinin sistemi yerel olarak ayağa kaldırması için gereken adım sayısını minimumda tutacak biçimde tasarlanmıştır.

Karşılaştırmalı değerlendirmede, geliştirilen sistemin ChatGPT, Gemini ve Copilot gibi ticari muadiller karşısında yanıt kalitesi açısından doğrudan kıyaslamaya tabi tutulması, kullanılan temel modelin (GPT-4o veya Claude Haiku 4.5) yetenekleriyle sınırlıdır. Asıl ayırt edici nokta, kullanıcının kendi verileri üzerinde sahiplik tutması ve sistemin yerel olarak çalıştırılabilmesidir. Bu yönüyle çalışma; üretkenlik asistanları pazarında "gizlilik ve veri sahipliği" başlığını öne çıkaran açık kaynak alternatiflere katkı sağlamaktadır.

Mimari ödünleşmeler açısından, MCP'nin tam ağ protokolünün (transport: stdio/SSE) gerçekleşmemesi bilinçli bir tercih olmuştur. Tek süreçli araç barındırma, ilk sürüm için dağıtım karmaşıklığını düşürmüş, ancak araçların başka istemci uygulamalarla paylaşılabilişliğini sınırlamıştır. Benzer biçimde, tek bir LLM sağlayıcısının kullanıcı başına seçilebilmesi, model arası geçişte sohbet bağlamının taşınması probleminin geleceğe bırakılmasını sağlamıştır.

Geliştirilen sistemin sınırlılıkları arasında en belirgin olanlardan biri, Tavily web arama aracı dışında, sistemin internet kaynaklı bilgi doğrulama mekanizmasına sahip olmamasıdır. Kullanıcı sorgusunda model yanlış bir bilgi üretirse (hallucination), sistem bu hatayı kendi başına tespit edememekte; yalnızca araç çağrılarının ürettiği yapılandırılmış sonuçları doğru biçimde işleyebilmektedir. Bu sınırlılık, üretim ortamına aktarımda model yanıtlarının kullanıcıya bir uyarı bandı ("AI tarafından üretilmiştir, kontrol ediniz") ile gösterilmesi yönünde bir tasarım kararı gereklidir.

Çalışmanın metodolojik katkısı açısından, geleneksel bir BT projesinden farklı olarak, sistemin kalbinin deterministik bir kural motoru değil, olasılıksal bir dil modeli olması, test stratejisinde de farklı bir yaklaşımı gerektirmiştir. Bölüm 6'da sunulan araç çağırma doğruluğu testleri, kabul kriterlerini ikili (geçti/kaldı) yerine yüzdesel ("%87 doğru kombinasyon") biçimde tanımlamayı zorunlu kılmaktadır. Bu metodoloji, sonraki yapay zekâ tabanlı sistemlerin sınamasında genel bir referans olarak kullanılabilir.

Sonuç bölümünde özetlenen sınırlılıklar, gelecekteki çalışmalar için net bir yol haritası sunmaktadır. Öneriler yalnızca özellik listesi olarak değil; her bir maddenin mevcut mimariye

entegrasyon maliyeti, beklenen kullanıcı değeri ve teknik risk düzeyi açısından değerlendirilmesi hedeflenmiştir.

Tam MCP protokol desteğinin getireceğı en belirgin kazanım, araçların bağımsız süreçler olarak ölçeklenebilmesidir. Mevcut monolitik registry yaklaşımı geliştirme hızını artırmakta; ancak her araç aynı Node.js sürecinde çalıştığından, tek bir entegrasyonun aşırı bellek tüketmesi tüm sistemi etkileyebilmektedir. MCP sunucusu modelinde bu risk izole edilebilir.

Redis tabanlı token önbellege, özellikle yoğun sohbet trafiğinde veritabanı okuma yükünü azaltacaktır. Her araç çağrısında oauth_accounts tablosundan şifre çözme işlemi tekrarlandığından, dakikada onlarca mesaj gönderen aktif kullanıcılarda bu maliyet birikimli hâle gelmektedir. Önbelleg stratejisinde TTL değerin token_expiry ile senkronize edilmesi ve kullanıcı oturumu kapandığında anahtarların geçersiz kılınması güvenlik açısından zorunlu görölmektedir.

Önerilerin uygulanma önceliğı, etki/efor matrisi ile sınıflandırılmıştır. Redis token önbellege ve CI/CD entegrasyonu düşük efor-yüksek etki kategorisindedir; tam MCP protokol desteğı ve çoklu bölge dağıtımı ise yüksek efor gerektirmektedir. Lisans tezi kapsamının ötesinde kalan maddeler, gelecekteki yüksek lisans veya endüstri iş birliğı projeleri için referans niteliğı taşımaktadır.

Kullanılabilirlik çalışması (SUS anketi) önerisi, sistemin akademik değerlendirmesini güçlendirecektir. On beş kişilik pilot grupta görev tamamlama süresi, hata sayısı ve subjektif memnuniyet ölçülerek arayüz iyileştirmelerine girdi sağlanabilir. Özellikle BYOK onboarding adımının SUS skoruna etkisi ayrıca analiz edilmelidir.

8. ÖNERİLER

Gelecek çalışmalar için şu öneriler ortaya konabilir:

1. **Tam MCP protokol desteği:** Araçlar ayrı bir MCP sunucusu olarak barındırılarak protokolün ağ tarafı gerçekleştirilebilir; böylece araçlar birden fazla AI ürünü tarafından paylaşılabilir hâle gelir.
2. **Token ön belleği:** Redis tabanlı bir in-memory ön bellek, token çözme maliyetini veritabanı çağrılarından uzaklaştırabilir.
3. **Yeni araç entegrasyonları:** Slack, Gmail (tüm MIME tipleri), Microsoft Graph (Outlook, OneDrive), Jira, Linear gibi araçlar entegre edilebilir.
4. **Kullanıcı başına çoklu provider:** Aynı kullanıcı için hem OpenAI hem Anthropic anahtarı saklanarak model bazlı geçiş desteklenebilir.
5. **Rol tabanlı erişim kontrolü:** Takım kullanımı için kullanıcı rolleri (admin, üye) ve paylaşılan bağlantılar eklenebilir.
6. **Kullanılabilirlik çalışması:** 10-15 kişilik pilot grup üzerinde System Usability Scale (SUS) değerlendirmesi yapılabilir.
7. **Gözlemlenebilirlik:** OpenTelemetry ile dağıtık izleme (tracing) ve Prometheus metrikleri eklenerek operasyonel olgunluk artırılabilir.

Yukarıdaki önerilerin yanı sıra, sistemin üretim olgunluğuna taşınması için aşağıdaki kalemler de önemlidir.

Sürekli Entegrasyon ve Sürekli Dağıtım (CI/CD): GitHub Actions üzerinden otomatik birim/entegrasyon testleri, Docker imaj derlemesi ve npm audit zafiyet taraması her pull request adımında çalıştırılmalıdır. Üretim ortamına aktarım için imzalı imajların kullanılması ve Cosign benzeri bir araçla doğrulanması güvenlik zincirini güçlendirecektir.

Çoklu Bölge Dağıtımı ve Yedekleme: Kullanıcı verisinin yer tuttuğu PostgreSQL veritabanı için günlük noktalı yedekleme (point-in-time recovery) etkinleştirilmelidir. Coğrafi yedek alma stratejisi, KVKK ve GDPR uyumluluğu açısından da değer taşımaktadır.

Anahtar Rotasyonu ve Sır Yönetimi: ENCRYPTION_KEY ortam değişkeni, üretim ortamında doğrudan dosyada tutulmak yerine HashiCorp Vault veya AWS Secrets Manager gibi sır yönetimi servislerinde tutulmalıdır. Yıllık rotasyon politikası uygulamaya alınarak şifre çözme yetkilendirme etiketinin (auth_tag) tekrar doğrulanması sağlanmalıdır.

Çoklu Modelli Yönlendirme: Hızlı sorgular için Claude Haiku 4.5, uzun bağlamlı görevler için GPT-4o gibi modelden modele yönlendiren bir adaptör katmanı, hem maliyet hem de yanıt kalitesi açısından optimize edilebilir.

Federe Araç Mağazası: Topluluk tarafından geliştirilen MCP araçlarının doğrulanmış imzalarla taşınabileceği bir merkezi kayıt servisi (registry), platformun ekosistem değerini artıracaktır.

Çok Dilli Arayüz Desteği: Mevcut sürümde arayüz dili İngilizce ve sistem mesajları Türkçe-İngilizce karışıktır. i18n altyapısı eklenerek, kullanıcı tercihinin yansıdığı ve hatta sistem mesajının dile göre uyarlandığı bir kişiselleştirme katmanı tasarlanabilir.

Çevrimdışı Çalışma ve Edge Deployment: Hassas kuruluşlar için, LLM sağlayıcısının yerel bir model (Llama 3, Mistral vb.) ile değiştirilebileceği ve sistemin internet bağlantısı gerektirmeden çalıştırılabileceği bir alt sürüm tanımlanabilir.

Bu öneriler, prototipin sınırlı kapsamından üretim ortamına geçişte karşılaşılmaması beklenen mühendislik ve operasyonel sorunlara yönelik bir yol haritası oluşturmaktadır. Çalışmanın açık kaynak deposunda, bu kalemler GitHub Issue etiketleriyle ileride takip edilecek biçimde belgelenmiştir.

KAYNAKLAR DİZİNİ

Anthropic, 2024a, *Introducing the Model Context Protocol*, Anthropic Blog, <https://www.anthropic.com/news/model-context-protocol> (Erişim tarihi: 21 Nisan 2026).

Anthropic, 2024b, *Tool Use (Function Calling) – Claude Documentation*, <https://docs.anthropic.com/en/docs/build-with-claude/tool-use> (Erişim tarihi: 21 Nisan 2026).

Google, 2024, *Using OAuth 2.0 to Access Google APIs*, Google Identity Platform, <https://developers.google.com/identity/protocols/oauth2> (Erişim tarihi: 21 Nisan 2026).

Google, 2026a, *Gemini*, <https://gemini.google.com/> (Erişim tarihi: 21 Nisan 2026).

Google, 2026b, *Gmail API Documentation*, <https://developers.google.com/gmail/api> (Erişim tarihi: 21 Nisan 2026).

Google, 2026c, *Google Calendar API Documentation*, <https://developers.google.com/calendar> (Erişim tarihi: 21 Nisan 2026).

Google, 2026d, *Google Drive API Documentation*, <https://developers.google.com/drive> (Erişim tarihi: 21 Nisan 2026).

Hardt, D., 2012, *The OAuth 2.0 Authorization Framework*, RFC 6749, Internet Engineering Task Force (IETF).

Mark, G., Gudith, D. and Klocke, U., 2008, The cost of interrupted work: More speed and stress, *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08)*, 107-110 pp.

Microsoft, 2026, *Microsoft Copilot*, <https://copilot.microsoft.com/> (Erişim tarihi: 21 Nisan 2026).

National Institute of Standards and Technology (NIST), 2007, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST Special Publication 800-38D.

Notion, 2024, *Authorization – Notion API*, Notion Developers, <https://developers.notion.com/docs/authorization> (Erişim tarihi: 21 Nisan 2026).

Open WebUI, 2026, *Open WebUI – User-Friendly AI Interface*, <https://github.com/open-webui/open-webui> (Erişim tarihi: 21 Nisan 2026).

OpenAI, 2023, *Function Calling and Other API Updates*, OpenAI Blog, <https://openai.com/blog/function-calling-and-other-api-updates> (Erişim tarihi: 21 Nisan 2026).

OpenAI, 2026, *ChatGPT*, <https://chat.openai.com/> (Erişim tarihi: 21 Nisan 2026).

PostgreSQL Global Development Group, 2026, *PostgreSQL 16 Documentation*, <https://www.postgresql.org/docs/16/> (Erişim tarihi: 21 Nisan 2026).

Provos, N. and Mazieres, D., 1999, *A future-adaptable password scheme*, USENIX Annual Technical Conference, 81-91 pp.

Tavily, 2026, *Tavily Search API Documentation*, <https://docs.tavily.com/> (Eriřim tarihi: 21 Nisan 2026).

Vercel, 2026, *Vercel AI SDK Documentation*, <https://sdk.vercel.ai/docs> (Eriřim tarihi: 21 Nisan 2026).